

運用 Google 代理堆疊建構多代理創意工作室： Cloud Run 和 Agent Runtime 的 ADK、A2A、 MCP

來源：<https://codelabs.developers.google.com/ai-creative-studio-adk-a2a?hl=zh-tw>

步驟數：16

使用 ADK、技能、A2A、MCP、Cloud Run 和 Gemini Enterprise Agent Platform Runtime，建構分散式多模態多代理系統

目錄

1. [總覽](#)
2. [設定環境](#)
3. [瞭解 Google ADK](#)
4. [建構及測試品牌策略師代理程式](#)
5. [建構文案人員 - ADK 技能](#)
6. [建構 Designer - 多模態圖像生成](#)
7. [建構 Critic - 結構化輸出內容](#)
8. [使用 MCP 建構專案經理代理](#)
9. [瞭解 A2A 通訊協定](#)
10. [將代理公開為 A2A 服務](#)
11. [建構創意總監自動調度管理工具](#)
12. [部署及測試專家代理程式](#)
13. [將創意總監部署至 Agent Runtime](#)
14. [執行端到端廣告活動](#)
15. [清除](#)
16. [摘要](#)

1. 總覽

從做中學 免付費

立即取得 Google Cloud 抵免額，並完成這個實驗室。無須提供信用卡或付款方式。

info

從做中學 免付費

立即取得 Google Cloud 抵免額，並完成這個實驗室。無須提供信用卡或付款方式。

啟用

如果參加**講師主講的研討會**：講師會提供抵免額代碼。請使用他們提供的連結。

在本程式碼研究室中，您將建構 **AI Creative Studio**，這是一個分散式多代理系統，可將單一提示詞轉換為完整的 Instagram 廣告活動。

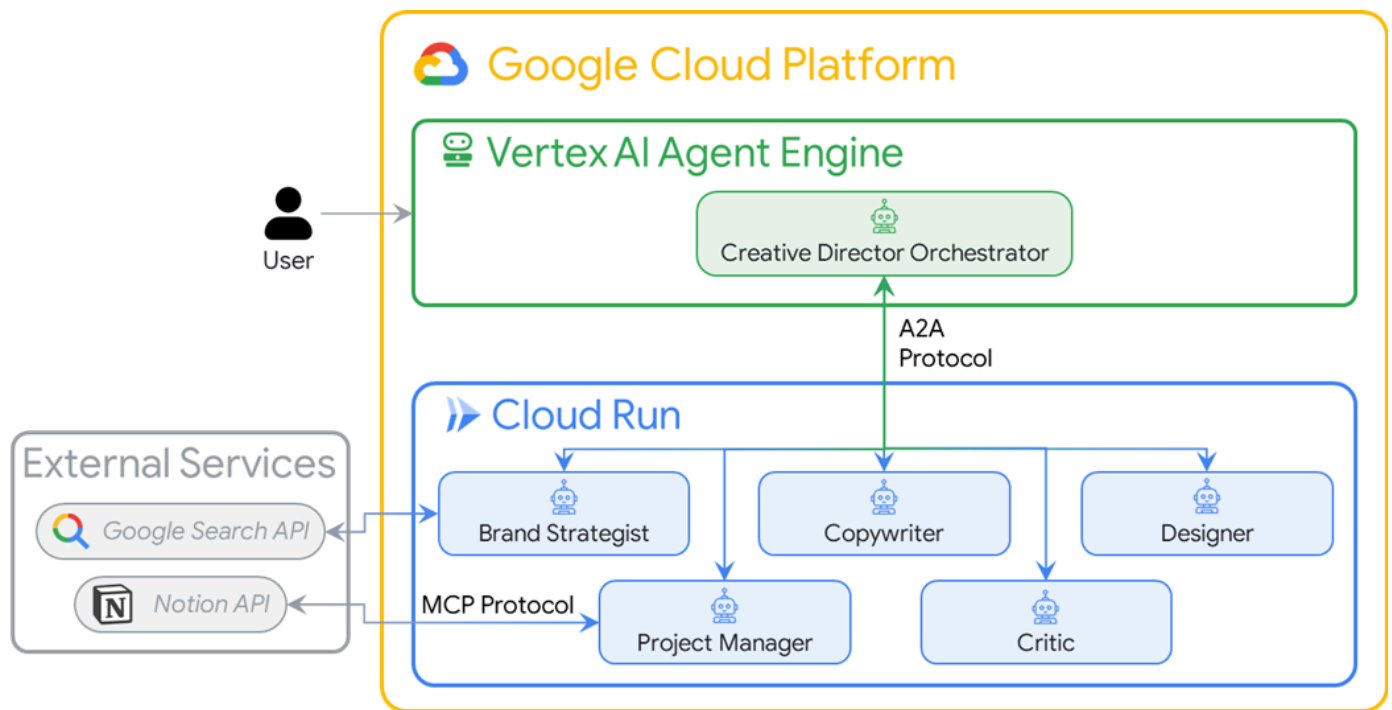
輸入一句話，取得觀眾研究、說明文字、視覺概念、經過品質審查的文案，以及完整的專案時間表，這些都是由 AI 代理團隊協作生成。

您要建構的代理

代理	角色
品牌策略規劃人員	在網路上搜尋目標對象洞察、競爭對手分析和 2025 年趨勢
文案撰寫	撰寫含主題標記和行動號召的 Instagram 說明文字 - 由 ADK 技能提供支援，可依需求載入平台指南和說明文字公式
設計人員	透過 Gemini 建立視覺概念並生成真實圖片，儲存在 GCS 中
評論家	評論副本和圖片 - 傳回 <code>APPROVED</code> 或 <code>NEEDS_REVISION</code> ，並提供具體意見回饋
專案經理	建立專案時間表和工作細目，並視需要透過 MCP 同步至 Notion
創意總監	依序調度所有五位專家 - 您只要提供一個提示，其餘工作都會由 AI 負責協調

這 5 個代理程式會部署為獨立的 **Cloud Run** 微服務。這些代理會透過 **A2A 通訊協定** 進行通訊。這項開放標準與語言無關，因此無論框架為何，任何代理都能呼叫其他代理。創意總監會在 **Agent Runtime** 上執行，並遠端連線至每位專員。

架構



課程內容

- 使用 **Google ADK** 建構 LLM 代理程式 - **Agent**、系統指令和內建工具。
- 使用 **ADK Skills** (**SkillToolset**), 將可重複使用的代理知識封裝至模組化檔案。
- 透過 **FunctionTool** 將文字代理程式連結至圖像模型, 生成真實圖像。
- 使用 **Model Context Protocol (MCP)** 整合外部 API, 不必編寫自訂膠合程式碼。
- 透過 HTTPS 使用 **Agent to Agent 通訊協定 (A2A)**, 將任何代理轉換為可透過網路呼叫的服務。
- 使用 **RemoteA2aAgent** 和 **AgentTool** 自動化調度管理分散式代理。
- 將獨立代理封裝為 **Cloud Run** 微服務並部署。
- 在 **Agent Runtime** 上代管有狀態的自動化調度管理工具。
- 使用**脈絡壓縮**功能, 將冗長的多代理工作流程控制在脈絡限制內。
- 建立**品質控管迴圈**: 評論家審查輸出內容 → 視需要自動修訂。

軟硬體需求

- 已啟用計費功能的 **Google Cloud 專案**
- 「擁有者」或「編輯者」IAM 角色
- Python 基礎知識

步驟 2 · 約 7 分鐘

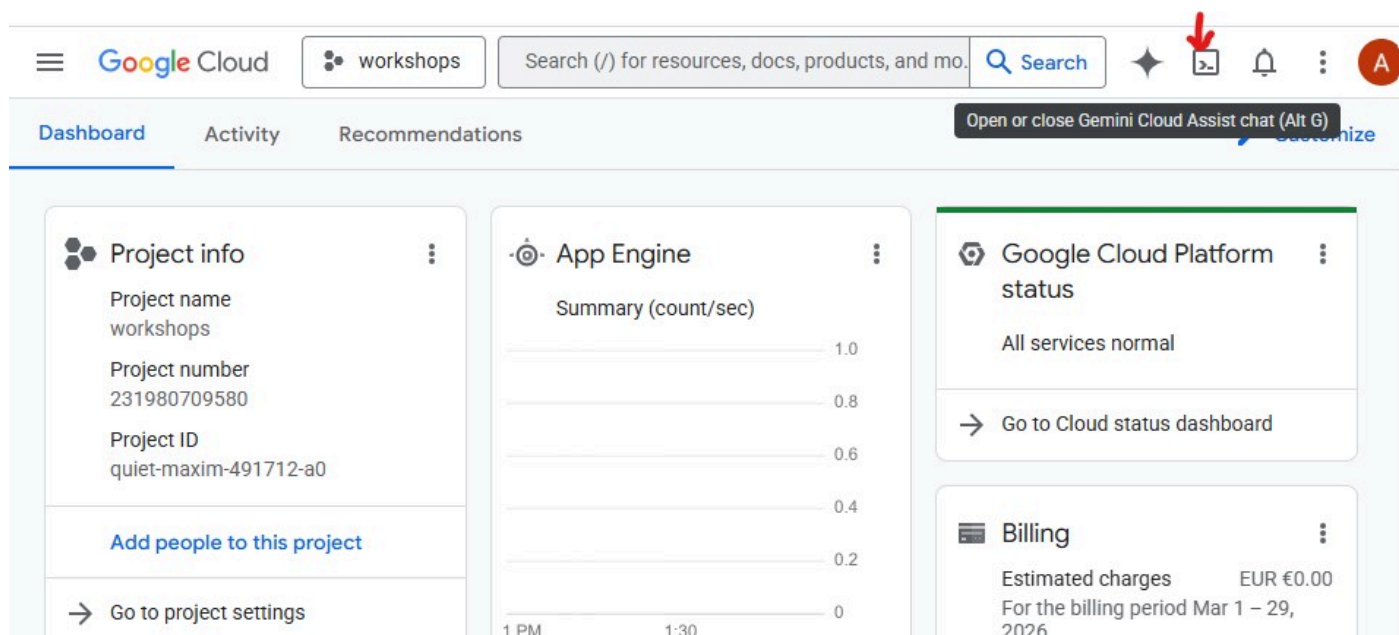
2. 設定環境

在本程式碼研究室中，我們將使用 Cloud Shell。

什麼是 Cloud Shell？

Cloud Shell 是免費的瀏覽器型 Linux 環境，已預先安裝 `gcloud`、`git`、Python、Docker 等工具。您不必在本機安裝任何項目。

若要開啟 Cloud Shell，請按一下 GCP 控制台右上工具列中的終端機圖示：



首次開啟 Cloud Shell 時，系統會提示您驗證帳戶，請按一下「驗證」：

The screenshot shows the Google Cloud Platform dashboard for a project named 'workshops'. The dashboard includes sections for Project info, App Engine, Google Cloud Platform status, and Billing. A modal window titled 'Verify your account' is overlaid on the dashboard, prompting the user to verify their account to start using Cloud Shell. The modal contains the text: 'Verify your account to start using Cloud Shell. Click refresh after verification to proceed.' and two buttons: 'Refresh' and 'Verify'. A red arrow points to the 'Verify' button. The background dashboard shows the following details:

- Project info:** Project name: workshops, Project number: 231980709580, Project ID: quiet-maxim-491712-a0. A link 'Add people to this project' is visible.
- App Engine:** Summary (count/sec) with a gauge showing values from 0.2 to 1.0.
- Google Cloud Platform status:** All services normal. A link 'Go to Cloud status dashboard' is visible.
- Billing:** Estimated charges: EUR €0.00.

The modal window also features a blue star icon and the text: 'Build with Gemini CL', 'Gemini Code Assist', and 'Ask Gemini in Cloud Shell Term Editor.'.

然後按一下「Authorize」(授權)，允許 Cloud Shell 發出 Google Cloud API 呼叫：

Google Cloud | workshops | Search (/) for resources, docs, products, and ... Search

Dashboard | Activity | Recommendations | Customize

Project info

Project name
workshops

Project number
231980709580

Project ID
quiet-maxim-491712-a0

[Add people to this project](#)

App Engine

Summary (count/sec)

	1.0
	0.8
	0.6
	0.4
	0.2

Google Cloud Platform status

All services normal

[Go to Cloud status dashboard](#)

Billing

Estimated charges EUR €0.00

Connecting... x +

[Open Editor](#)

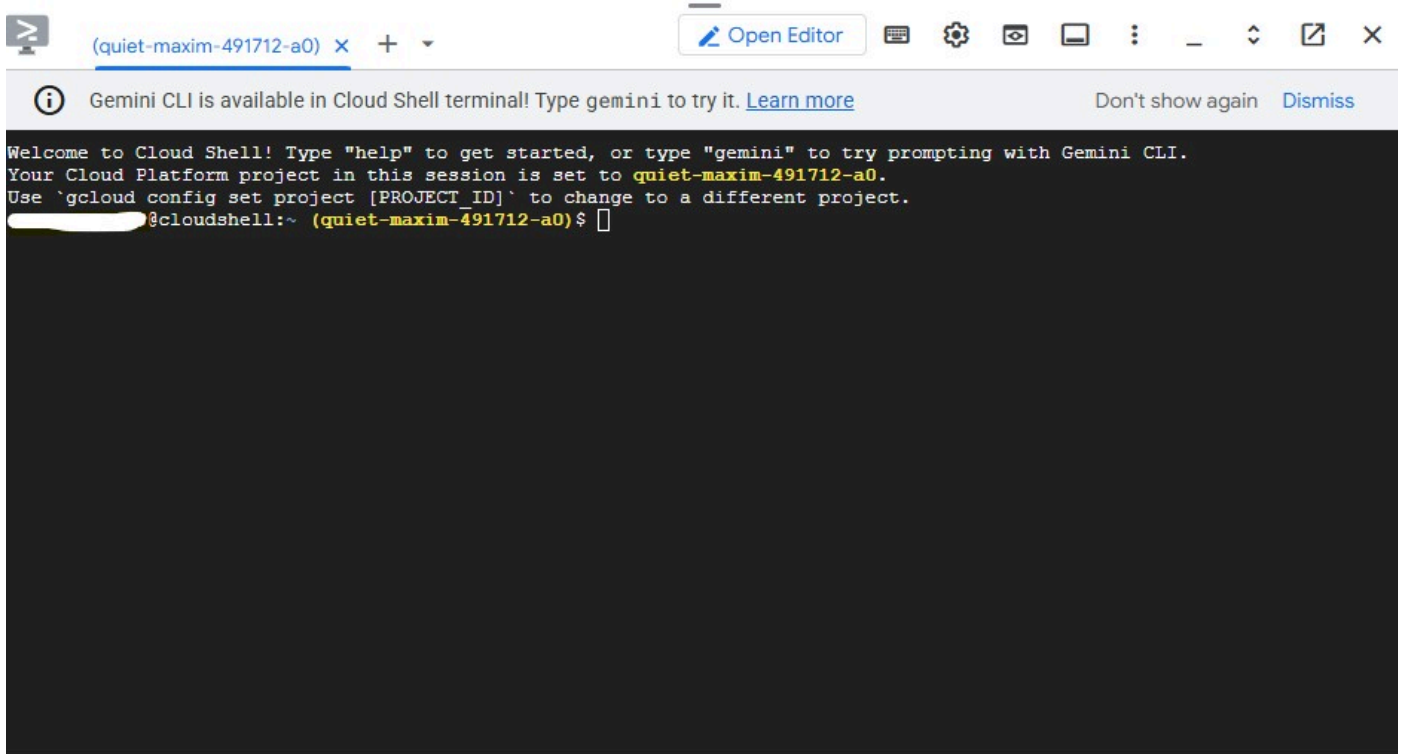
Authorize Cloud Shell

Cloud Shell needs permission to use your credentials to make Google Cloud API calls.

Click Authorize to grant permission to this and future calls.

[Reject](#) [Authorize](#)

Cloud Shell 現已準備就緒。終端機中會顯示歡迎訊息：



```
(quiet-maxim-491712-a0) x + v
Open Editor
Gemini CLI is available in Cloud Shell terminal! Type gemini to try it. Learn more
Don't show again Dismiss
Welcome to Cloud Shell! Type "help" to get started, or type "gemini" to try prompting with Gemini CLI.
Your Cloud Platform project in this session is set to quiet-maxim-491712-a0.
Use `gcloud config set project [PROJECT_ID]` to change to a different project.
@cloudshell:~ (quiet-maxim-491712-a0)$
```

注意：Cloud Shell 閒置 20 分鐘後就會中斷連線。如果發生這種情況，請重新連線並輸入 `cd ~/ai-creative-studio/workshop/starter` 返回工作目錄。

驗證及設定專案

Cloud Shell 已透過您的 Google 帳戶完成驗證。確認有效帳戶並找出專案 ID：

```
gcloud config list
```

您也可以 GCP 主控台資訊主頁的左側面板中，查看專案 ID。複製該路徑，下一個指令會用到：

The screenshot shows the Google Cloud Platform console interface. On the left, the 'Project info' panel displays the project name 'workshops', project number '231980709580', and project ID 'quiet-maxim-491712-a0'. The 'App Engine' panel shows a summary of instances per second (count/sec) with a scale from 0.2 to 1.0. The 'Google Cloud Platform status' panel indicates 'All services normal'. Below the console, a Cloud Shell terminal window is open, showing the command 'gcloud auth list' and its output, which lists the active account. The terminal also shows the command 'export PROJECT_ID=quiet-maxim-491712-a0' being executed.

Project info

Project name
workshops

Project number
231980709580

Project ID
quiet-maxim-491712-a0

Add people to this project

App Engine

Summary (count/sec)

1.0

0.8

0.6

0.4

0.2

Google Cloud Platform status

All services normal

Go to Cloud status dashboard

Billing

Estimated charges

Open Editor

Gemini CLI is available in Cloud Shell terminal! Type gemini to try it. [Learn more](#)

Welcome to Cloud Shell! Type "help" to get started, or type "gemini" to try prompting with Gemini CLI. Your Cloud Platform project in this session is set to quiet-maxim-491712-a0. Use 'gcloud config set project [PROJECT_ID]' to change to a different project.

@cloudshell:~ (quiet-maxim-491712-a0)\$ gcloud auth list

Credentialed Accounts

ACTIVE: *

ACCOUNT: [\[redacted\]](#)

To set the active account, run:

\$ gcloud config set account 'ACCOUNT'

@cloudshell:~ (quiet-maxim-491712-a0)\$ export PROJECT_ID=quiet-maxim-491712-a0

現在設定專案：

```
export PROJECT_ID=$(gcloud config get-value project)
export REGION="us-central1"          # Cloud Run deployment region
echo "Project: $PROJECT_ID"
```

預期輸出內容：

```
Project: my-project-123
```

啟用必要的 API

```
gcloud services enable \  
  aiplatform.googleapis.com \  
  apphub.googleapis.com \  
  run.googleapis.com \  
  cloudbuild.googleapis.com \  
  artifactregistry.googleapis.com \  
  generativelanguage.googleapis.com \  
  iam.googleapis.com \  
  cloudresourcemanager.googleapis.com \  
  storage.googleapis.com \  
  secretmanager.googleapis.com
```

整個流程大約需要 2 分鐘。完成後，你會看到 `Operation finished successfully`。

設定應用程式預設憑證 (ADC)

代理程式會使用 **Google Auth** 程式庫呼叫 Gemini Enterprise Agent Platform，這需要應用程式預設憑證，與 `gcloud` CLI 驗證不同。

執行一次：

```
gcloud auth application-default login
```

瀏覽器分頁隨即開啟，並要求您確認。點按「允許」，您會看到：

```
Credentials saved to file: ~/.config/gcloud/application_default_credentials.json
```

為什麼需要授予存取權？`gcloud auth login` 驗證 CLI。`gcloud auth application-default login` 會建立憑證，供 Python 應用程式 (例如代理程式) 用來呼叫 Google Cloud API。如果沒有這個屬性，代理程式就會失敗並顯示 `service account info is missing 'email' field`。

複製範例存放區

本程式碼研究室會使用入門存放區，也就是包含所有基礎架構 (Dockerfile、pyproject.toml、部署指令碼) 的骨架專案，但代理程式邏輯則由您編寫。

```
git clone https://github.com/Saoussen-CH/mas-a2a-gcp.git ~/ai-creative-studio  
cd ~/ai-creative-studio/workshop/starter
```

每個 `agent.py` 都包含 `# TODO` 預留位置，您可以在其中編寫代理程式邏輯。`Dockerfile`、`pyproject.toml` 和部署指令碼已完成。

設定環境變數

複製提供的範例，並在一個步驟中插入專案 ID：

```
cp .env.example .env
sed -i "s|GOOGLE_CLOUD_PROJECT=your-project-id|GOOGLE_CLOUD_PROJECT=$(gcloud config get-value project)|" .env
```

接著建立 GCS bucket，Designer 會將生成的圖片儲存在其中，並以 bucket 名稱更新 `.env`：

```
export PROJECT_ID=$(gcloud config get-value project)
export BUCKET_NAME="${PROJECT_ID}-campaign-images"

gcloud storage buckets create gs://${BUCKET_NAME} \
  --location=us-central1 \
  --project=${PROJECT_ID}

sed -i "s|GCS_IMAGES_BUCKET=your-project-id-campaign-images|GCS_IMAGES_BUCKET=${BUCKET_NAME}|" .env
```

然後設定經簽署的圖片網址支援功能。創意總監會為最終廣告活動摘要中的每張圖片產生可點選的 HTTPS 連結。這需要服務帳戶簽署網址。請執行下列指令進行設定：

```
export PROJECT_NUMBER=$(gcloud projects describe $(gcloud config get-value project) --format="value(projectNumber)")
export SA_EMAIL="${PROJECT_NUMBER}-compute@developer.gserviceaccount.com"
export AGENT_RUNTIME_SA="service-${PROJECT_NUMBER}@gcp-sa-aipatform-re.iam.gserviceaccount.com"

# Allow your user account to sign URLs locally (adk web)
gcloud iam service-accounts add-iam-policy-binding ${SA_EMAIL} \
  --member="user:$(gcloud config get-value account)" \
  --role="roles/iam.serviceAccountTokenCreator"

# Allow Agent Runtime to sign URLs when deployed
gcloud projects add-iam-policy-binding $(gcloud config get-value project) \
  --member="serviceAccount:${AGENT_RUNTIME_SA}" \
  --role="roles/iam.serviceAccountTokenCreator"

# Save SA email and project number to .env
grep -q "^SIGNING_SERVICE_ACCOUNT" .env \
  && sed -i "s|^SIGNING_SERVICE_ACCOUNT=.*|SIGNING_SERVICE_ACCOUNT=${SA_EMAIL}|" .env \
  || echo "SIGNING_SERVICE_ACCOUNT=${SA_EMAIL}" >> .env

grep -q "^GOOGLE_CLOUD_PROJECT_NUMBER" .env \
  && sed -i
"s|^GOOGLE_CLOUD_PROJECT_NUMBER=.*|GOOGLE_CLOUD_PROJECT_NUMBER=${PROJECT_NUMBER}|" .env \
  || echo "GOOGLE_CLOUD_PROJECT_NUMBER=${PROJECT_NUMBER}" >> .env
```

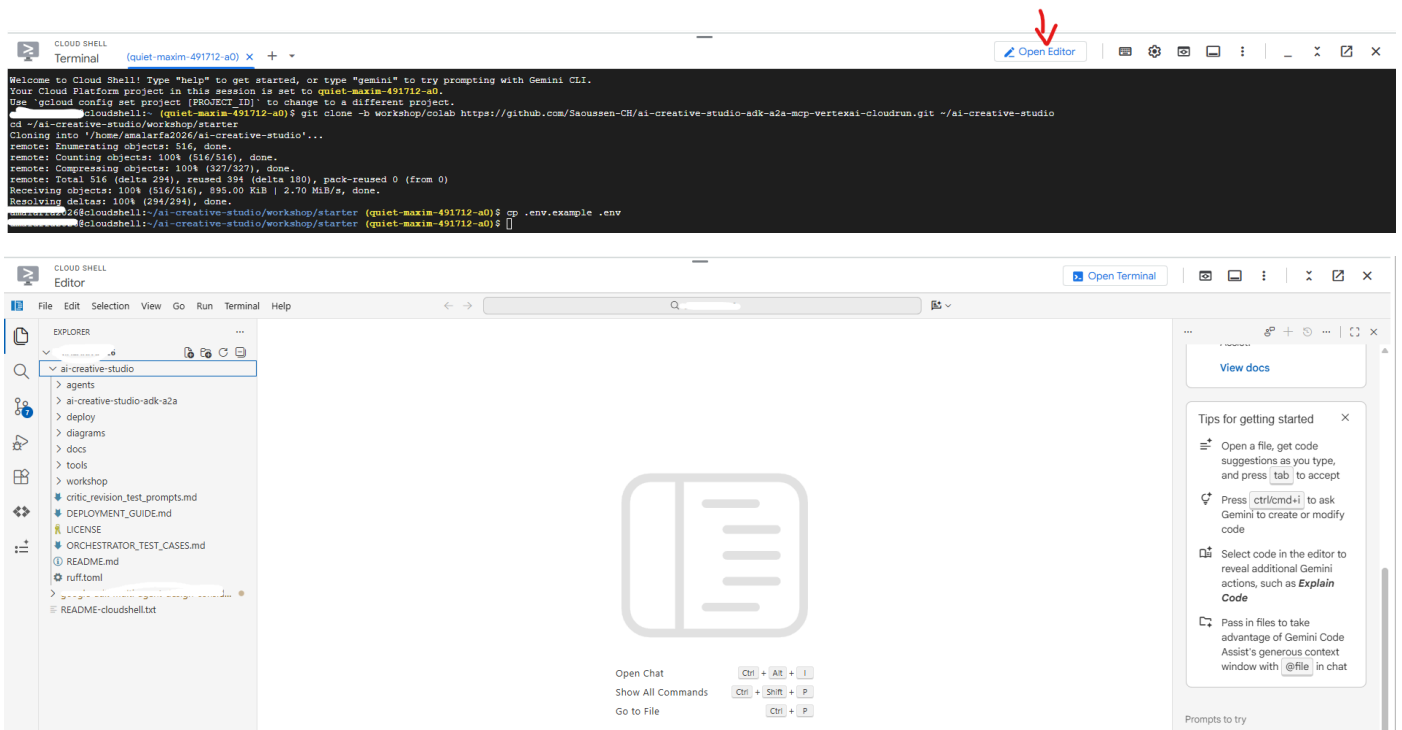
為什麼要使用 **GCS bucket**？設計工具會使用 Gemini 生成圖片，並上傳至 GCS。在 A2A 中傳回原始圖片位元組並不實際，因為 `gs://` URI 是輕量指標，任何代理程式都可以參照。Critic 會收到 URI 並傳遞至 Vertex AI，後者會在伺服器端擷取圖片，Critic 容器完全不會直接觸及 Cloud Storage。

注意： `.env` 是隱藏檔案 (開頭為 `.`)。執行 `ls` 不會顯示這個檔案，請使用 `ls -a` 確認檔案是否已建立。Cloud Shell 編輯器也會在檔案樹狀結構中隱藏點檔案，因此請一律直接從終端機開啟 `.env`。

在編輯器中開啟 `.env`，即可查看所有設定：

```
cloudshell edit .env
```

這會在 Cloud Shell 編輯器中以分頁的形式開啟 `.env`。如果編輯器面板未顯示，請點選工具列中的「開啟編輯器」按鈕：



確認專案設定正確無誤：

```
grep GOOGLE_CLOUD_PROJECT .env
```

安裝依附元件

我們使用 `uv`，這是一種快速的現代 Python 套件管理工具，可處理虛擬環境並安裝在單一工具中。速度比 `pip` 快約 10 到 100 倍，是管理 Python 專案的建議方式。

Cloud Shell 已安裝 `uv`。所有代理程式共用相同的核心依附元件，因此安裝一次即可，適用於本程式碼研究室中的每個代理程式：

```
uv sync
```

`uv sync` 指令會讀取 `pyproject.toml`，並建立包含所有依附元件的 `.venv/` 目錄。每位專家也有專供 Docker 建構作業使用的 `pyproject.toml`，上述共用安裝作業涵蓋本機測試所需的一切。

無須手動啟用。本程式碼研究室中的所有指令都會使用 `uv run`，這會自動在 `~/ai-creative-studio/workshop/starter/` 中啟用 `.venv`。只要 `cd` 到 `starter` 目錄，並在指令前加上 `uv run` 即可。

3. 瞭解 Google ADK

撰寫程式碼前，請先瞭解 **Agent Development Kit (ADK)**，也就是您在本程式碼實驗室中建構每個代理時使用的框架。

什麼是 ADK？

Agent Development Kit (ADK) 是彈性十足的模組化框架，可用於開發及部署 AI 代理。雖然 ADK 已針對 Gemini 和 Google 生態系統最佳化，但不受模型和部署方式限制，且可與其他架構相容。ADK 的設計宗旨是讓代理開發更貼近軟體開發，方便開發人員建立、部署及自動調度管理代理架構，無論是簡單的工作或複雜的工作流程，都能輕鬆完成。

ADK 會處理複雜的部分 (工具呼叫、多輪對話、情境管理、串流)，讓您專注於代理邏輯。

ADK 代理的構成元素

每個代理程式都由四個建構區塊組成：

封鎖	角色
型號	負責理解目標、擬定計畫並生成回覆的 LLM
工具	透過呼叫 API 或服務擷取資料或執行動作的函式
自動化調度管理	在回合之間保留記憶和狀態、傳送工具呼叫、將結果傳回模型
執行階段	在叫用時執行系統 - 本機透過 <code>adk web</code> 執行，或以部署的服務執行

代理程式定義

本程式碼研究室中的 5 個代理程式定義方式都相同：

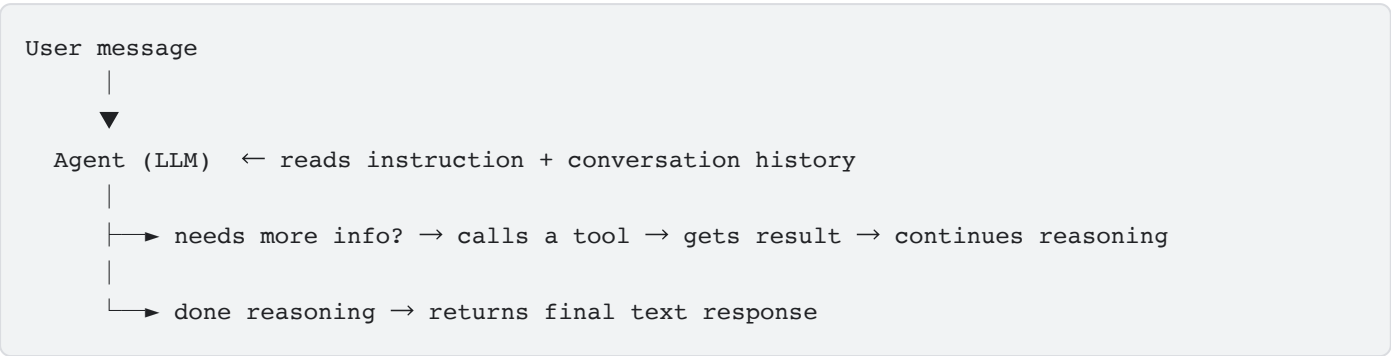
```
from google.adk.agents import Agent
from google.adk.tools.google_search_tool import google_search

root_agent = Agent(
    name="brand_strategist", # unique identifier
    model=os.getenv("GEMINI_MODEL", "gemini-2.5-flash"), # the LLM powering this agent
    instruction=SYSTEM_INSTRUCTION, # the agent's persona, constraints,
    and output format
    description="Brand strategist for market research, trend analysis, and competitive
insights",
    tools=[google_search], # functions the LLM can call
)
```

欄位	目的
----	----

name	專屬 ID - 用於將呼叫路徑傳送給協調器
model	支援這個代理程式的 Gemini 模型
instruction	系統提示 - 定義代理的角色、限制和輸出格式
description	單行摘要 - 自動調度管理工具會讀取這項資訊，決定要呼叫哪位專員
tools	大型語言模型可叫用的函式 (內建函式 (例如 google_search) 或自訂 Python 函式)

ADK 執行代理的方式



LLM 會自行決定是否呼叫工具、呼叫哪個工具，以及使用哪些引數。您只需撰寫指令，其餘工作就交給 ADK。

步驟 4 · 約 8 分鐘

4. 建構及測試品牌策略師代理程式

首先介紹第一個代理：品牌策略師。這個代理程式僅供研究，可使用 Google 搜尋尋找目標對象洞察資料、競爭對手分析和熱門主題。

在 Cloud Shell 編輯器中開啟骨架代理程式檔案：

```
cloudshell edit agents/brand_strategist/agent.py
```

畫面上會顯示兩個 `# TODO` 區段，供你填寫。

TODO 1 - Write the system instruction

首先，請為代理程式撰寫系統指令。系統指令是定義代理程式角色、限制和輸出格式的字串。

```
SYSTEM_INSTRUCTION = f"""You are a Brand Strategist specializing in market research and trend analysis.
```

```
IMPORTANT: Today's date is {datetime.date.today().strftime("%B %d, %Y")}.
When conducting research, focus on current trends from {datetime.date.today().year}.
Use search queries like "[topic] trends {datetime.date.today().year}" for recent insights.
```

```
IMPORTANT: Your role is RESEARCH ONLY. You do NOT create campaign content, captions, or designs.
```

```
After providing research insights, your work is complete.
```

```
Your expertise:
```

- Identifying target audience insights and behaviors
- Analyzing competitor strategies
- Researching current social media trends
- Understanding platform algorithms and best practices

```
You have access to:
```

- google_search: Search the web for competitors, trends, and market insights

```
When given a campaign brief:
```

1. Use google_search to research the target audience's current interests
2. Search for and analyze 2-3 competitor brands
3. Identify 3-5 trending topics related to the product category
4. Provide high-level strategic insights - NOT specific campaign content

```
DO NOT create captions, copy, designs, or any campaign content.
```

```
Format your output as:
```

```
**Audience Insights:**
```

```
[Key behaviors and preferences based on research]
```

```
**Competitive Analysis:**
```

```
[What 2-3 competitors are doing - strengths and weaknesses]
```

```
**Trending Topics:**
```

```
[3-5 relevant trends to consider]
```

```
**Key Strategic Insights:**
```

```
[High-level themes and positioning opportunities]
```

```
"""
```

待辦事項 2 - 建立 root_agent

接著，請將不完整的 `root_agent` 替換為：

```
root_agent = Agent(
    name="brand_strategist",
    model=os.getenv("GEMINI_MODEL", "gemini-2.5-flash"),
    instruction=SYSTEM_INSTRUCTION,
    description="Brand strategist for market research, trend analysis, and competitive insights",
    tools=[google_search],
)
```

使用 ADK 網頁介面在本機測試

現在，我們來使用 **ADK 網頁 UI** 測試代理程式。這是內建的對話介面，可供您在部署至雲端前測試代理程式。

```
uv run adk web agents --allow_origins='*'
```

為什麼 `--allow_origins='*'`？Cloud Shell 的網頁預覽功能會透過 `*.cloudshell.dev` 網址將本機伺服器設為 Proxy，這與 `127.0.0.1` 的來源不同。如果沒有這個標記，建立工作階段時，CORS 會封鎖開發人員 UI，您會看到 `403 Forbidden` 錯誤。這個標記會將所有來源加入許可清單，適合用於本機開發作業。

「No agents found in current folder」(目前資料夾中找不到代理程式)？請確認您是從 `~/ai-creative-studio/workshop/starter` 執行指令 (而非從 `agents/` 內執行)。 `agents` 引數會告知 `adk web` 要搜尋的位置。

您會看到：

```
INFO: Started server process
INFO: Uvicorn running on http://localhost:8000
```

伺服器現在會在 Cloud Shell 中執行：

如要在瀏覽器中開啟，請使用「網頁預覽」：

1. 查看頁面頂端的 **Cloud Shell 工具列**
2. 按一下「網頁預覽」圖示 (看起來像一個方塊，內含向上箭頭，位於 Cloud Shell 工具列的右上角)
3. 按一下「變更通訊埠」並輸入 `8000`，然後按一下「變更並預覽」

新的瀏覽器分頁會隨即開啟，顯示 ADK 網頁 UI。按一下左上角的「Select an agent」下拉式選單，即可查看所有代理：

選擇 `brand_strategist` 開始測試：

試試下列測試提示

在 ADK 網頁版 UI 的對話方塊中，嘗試：

- `Research the eco-friendly water bottle market for health-conscious millennials`
- `What are the top Instagram trends in the wellness space in 2025?`

您應該會看到代理程式呼叫 Google 搜尋，並傳回包含「目標對象洞察」、「競爭分析」和「熱門主題」部分的結構化研究結果。

步驟 5 · 約 3 分鐘

5. 建構文案人員 - ADK 技能

角色：將品牌研究結果轉化為 Instagram 說明文字。文案撰稿人會建立 3 種不同語氣的說明文字 (鼓舞人心、教育、社群)，每種都附上主題標記和行動號召。

概念：ADK 技能

如果採用簡單的做法，就是直接在系統提示中嵌入所有平台知識，包括字元限制、主題標籤等級、說明公式和品牌語氣範例。這樣做雖然可行，但會導致每次要求都包含代理程式偶爾才需要的內容。

ADK Skills (`SkillToolset`，ADK 1.25.0 中推出) 可讓您將知識封裝至模組化檔案，並提供三種載入層級：

- **L1 - 前言** (`name` + `description` in `SKILL.md`)：一律提供，用於技能探索
- **L2 - 指令** (`SKILL.md` 主體)：代理觸發技能時載入
- **L3 - 資源** (`references/` 和 `assets/` 檔案)：只有在代理程式明確讀取時才會載入

系統指令會縮減為簡短的角色陳述，外加「load the skill before writing」(先載入技能再撰寫)。只有在代理程式實際需要時，平台詳細資料才會進入內容視窗。

文案撰寫者技能位於 `agents/copywriter/skills/instagram-copywriting/`：

```
skills/  
  instagram-copywriting/  
    SKILL.md                ← L1 frontmatter (discovery) + L2 instructions (loaded on  
trigger)  
    references/  
      platform-guide.md      ← L3: character limits, hashtag tiers, algorithm signals  
      caption-formulas.md    ← L3: hook formulas, CTA patterns, full caption  
  structures  
    assets/  
      brand-voice-examples.md ← L3: annotated real-world caption examples
```

直接在 Cloud Shell 編輯器中開啟檔案：

```
cloudshell edit agents/copywriter/agent.py
```

待辦事項 1 - 匯入 `load_skill_from_dir` 和 `skill_toolset`

找到註解 `# TODO 1: Import load_skill_from_dir and skill_toolset`，並新增兩個匯入項目：

```
from google.adk.skills import load_skill_from_dir  
from google.adk.tools import skill_toolset
```

TODO 2 - Load the skill and create a SkillToolset

在匯入項目下方找到這兩則註解：

```
# TODO 2: Load the instagram-copywriting skill from the skills/ directory  
# TODO 2: Create a SkillToolset with the loaded skill
```

並替換為：

```
_instagram_skill = load_skill_from_dir(  
    pathlib.Path(__file__).parent / "skills" / "instagram-copywriting"  
)  
_copywriting_skills = skill_toolset.SkillToolset(skills=[_instagram_skill])
```

`load_skill_from_dir` 會讀取 `SKILL.md`，以及 `references/` 和 `assets/` 中的所有檔案。`SkillToolset` 會將其包裝成 ADK 代理程式可接受的格式，也就是工具集，而非原始技能。

TODO 3 - Register the toolset with the agent

找出 `tools=[]`，# TODO 3: Add the SkillToolset here 並替換為：

```
tools=[_copywriting_skills],
```

開啟技能檔案，查看結構：

```
cloudshell edit agents/copywriter/skills/instagram-copywriting/SKILL.md
```

讓 ADK 網頁 UI 保持執行狀態。使用代理程式下拉式選單切換至 `copywriter`，不必重新啟動伺服器。

如果沒有執行，請重新啟動：

```
uv run adk web agents --allow_origins='*'
```

試試看：將下拉式選單切換為 `copywriter`，然後傳送：

```
You are writing captions for EcoFlow Smart Water Bottle targeting health-conscious  
millennials aged 25-35.  
Audience insight: they prioritize sustainability, track health metrics, and share lifestyle  
content.  
Competitor insight: Hydro Flask dominates with lifestyle branding; S'well leads on premium  
aesthetics.  
Write 3 Instagram captions - one inspirational, one educational, one community-focused.  
Include 5 hashtags each and a CTA.
```

沒有共用回憶集錦。請注意，您必須手動貼上目標對象和競爭對手洞察資料，否則文案人員不會知道品牌策略師製作了什麼內容，除非協調人員明確轉寄。在多代理工作流程中，背景資訊傳遞完全由自動化調度管理工具負責。這是創意總監的工作，您將在下一個步驟中建構。

步驟 6 · 約 5 分鐘

6. 建構 Designer - 多模態圖像生成

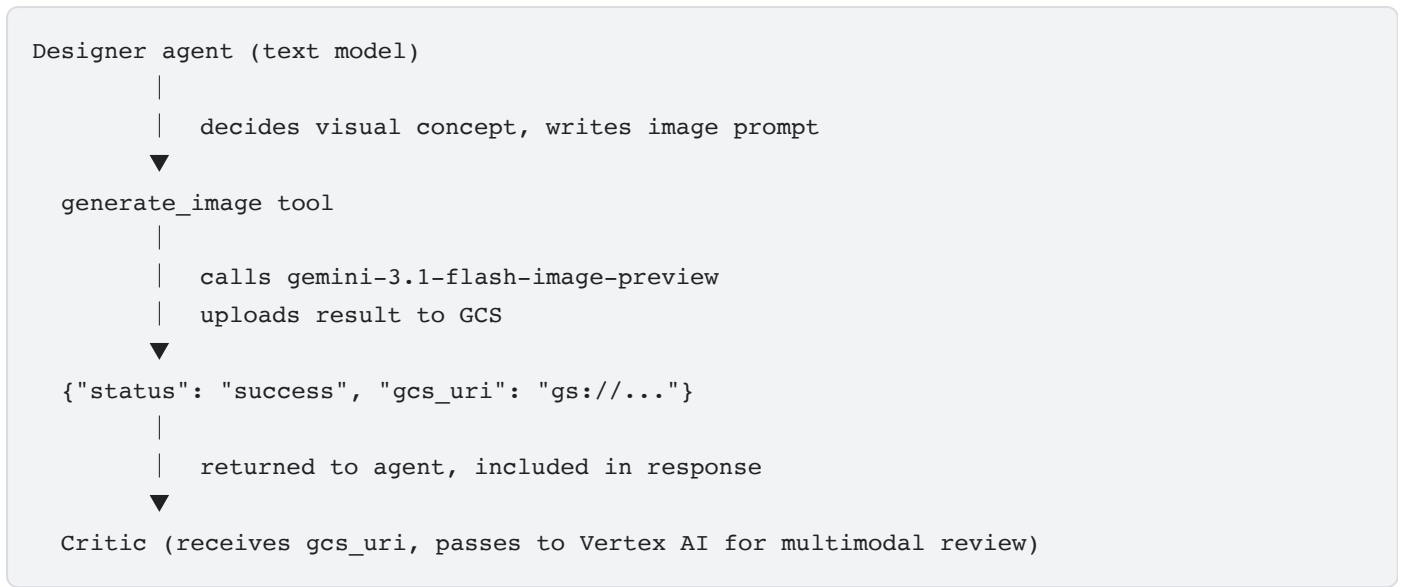
讓 ADK 網頁 UI 保持執行狀態。使用代理下拉式選單切換代理，不必重新啟動伺服器。

角色：為每個說明文字建立視覺概念，並使用 Gemini 原生圖像生成功能生成實際圖像。設計師會根據每則說明輸出 1 個視覺概念，並提供詳細提示、風格、色調、情緒和 Instagram 格式，然後立即呼叫 `generate_image` 工具來生成實際圖片，並上傳至 GCS。

概念：透過工具連結文字代理程式與圖像模型

Designer 會在 `gemini-3-flash-preview` 上執行 (透過 `.env` 中的 `GEMINI_MODEL` 設定的文字模型)，但圖像生成需要專用模型 (`gemini-3.1-flash-image-preview`)。該圖片模型不支援函式呼叫，因此無法直接做為 ADK 代理。而是包裝在純 Python 函式中，並註冊為 `FunctionTool`。

這是 LLM 無法直接呼叫的任何模型或 API 的模式：將其包裝在工具中，讓代理程式協調何時呼叫，並取得結構化結果。



直接在 Cloud Shell 編輯器中開啟檔案：

```
cloudshell edit agents/designer/image_gen_tool.py
```

並提供函式簽章、環境設定和顯示比例插入作業。依序完成以下三個 TODO：

TODO 1 - Call the Gemini image model

找出 `# TODO 1` 註解，並替換為：

```

client = genai.Client(vertexai=True, project=project_id, location=location)

response = client.models.generate_content(
    model=image_model,
    contents=prompt_with_aspect,
    config=types.GenerateContentConfig(
        response_modalities=["IMAGE", "TEXT"],
        http_options=types.HttpOptions(
            retry_options=types.HttpRetryOptions(
                attempts=5, exp_base=2, initial_delay=30,
                http_status_codes=[429, 500, 503, 504],
            ),
            timeout=180_000,
        ),
    ),
)

```

為什麼 `initial_delay=30` 要這麼做？與文字模型相比，動態共用配額的圖像生成配額恢復速度較慢。30 秒的基礎延遲時間可讓共用集區在重試之間有時間重新填入。文字模型重試設定使用 `initial_delay=5`，這不足以支援圖片配額。

TODO 2 - Extract image bytes from the response

找出 `# TODO 2` 註解，並替換為：

```

image_bytes = None
mime_type = "image/png"
for part in response.candidates[0].content.parts:
    if part.inline_data is not None:
        image_bytes = part.inline_data.data
        mime_type = part.inline_data.mime_type or "image/png"
        break

if not image_bytes:
    return {"status": "error", "error": "Gemini returned no image data"}

```

TODO 3 - Upload to GCS and return the URI

找出 `# TODO 3` 註解，並替換為：

```
ext = "jpg" if "jpeg" in mime_type else "png"
from google.cloud import storage
gcs_client = storage.Client(project=project_id)
bucket = gcs_client.bucket(bucket_name)
blob_name = f"campaign-images/{concept_name}-{uuid.uuid4().hex[:8]}.{ext}"
blob = bucket.blob(blob_name)
blob.upload_from_file(io.BytesIO(image_bytes), content_type=mime_type)
gcs_uri = f"gs://{bucket_name}/{blob_name}"
```

試試看：將下拉式選單切換為 **designer**，然後傳送：

Create a visual concept and generate the image for an EcoFlow Smart Water Bottle Instagram post targeting health-conscious millennials.
Style: clean, modern, lifestyle-focused. Include a detailed prompt with color palette, mood, and format (1080x1080 or 1080x1350).

圖片會內嵌在 **adk web** 中。上傳至 GCS 後，**generate_image** 也會透過 **tool_context.save_artifact(...)** 將圖片位元組儲存為 ADK 構件。adk web 伺服器會使用記憶體內構件服務，直接在即時通訊 UI 中將這些內容顯示為縮圖。Cloud Run 沒有構件服務，因此呼叫會引發 **ValueError**，工具會擷取並忽略該呼叫，無論如何都會傳回 **gcs_uri**。

7. 建構 Critic - 結構化輸出內容

角色：在文案和圖片交給專案經理前，先確保品質。Critic 會評估這兩項交付項目，並傳回 `APPROVED` 或 `NEEDS_REVISION`，以及具體建議。如果輸入內容中含有 `gcs_uri` 值，系統會呼叫 `review_image` 工具，在評分前檢查每張生成的圖片。

概念：何時使用 Pydantic 模型處理 Gemini 輸出內容

這項規則與輸出內容的消費者有關：

- **Python 程式碼**會耗用這項資料：請使用 `response_schema` + Pydantic。程式碼無法處理模稜兩可的內容，因此您需要有結構保證，才能可靠地擷取欄位。
- **LLM**會使用這項資訊：文字格式 + 系統指令就足夠。LLM 可理解格式設定規則，並容許變異。

在 `review_image` 中，Python 程式碼需要 `score`、`approval_status`、`what_works`、`issues` 和 `suggestions` 做為型別值。傳遞 `response_schema=_GeminiReview` 會在 API 層級限制 Gemini 傳回有效的 JSON；`model_validate_json()` 會將其剖析為程式碼可穩定使用的型別物件。

```
class _GeminiReview(BaseModel):
    score: int = Field(ge=1, le=10)
    approval_status: Literal["APPROVED", "NEEDS_REVISION"]
    what_works: str
    issues: str
    suggestions: str
```

對比：創意總監 (另一個 LLM) 會閱讀評論家的最終評論。系統指令中強制執行的文字格式就足夠。如果消費者是 LLM，而非程式碼，則不需要結構定義。

直接在 Cloud Shell 編輯器中開啟檔案：

```
cloudshell edit agents/critic/image_review_tool.py
```

系統會提供 Pydantic 模型和提示。依序完成以下三個 TODO：

TODO 1 - Create an image part from the GCS URI

找出 `# TODO 1` 註解，並替換為：

```
image_part = types.Part.from_uri(file_uri=gcs_uri, mime_type=mime_type)
```

透過 GCS 進行多模態檢閱。 `Part.from_uri()` 會將圖片做為 GCS 參照傳遞。Vertex AI 會在伺服器端擷取圖片，Critic 容器絕不會直接下載圖片，因此 Critic 服務不需要 Cloud Storage 憑證。

TODO 2 - Call Gemini with a structured response schema

找出 `# TODO 2` 註解，並替換為：

```
response = client.models.generate_content(
    model=model,
    contents=[image_part, prompt],
    config=types.GenerateContentConfig(
        response_schema=_GeminiReview,
        response_mime_type="application/json",
    ),
)
```

透過 **Pydantic** 輸出結構化內容。傳遞 `response_schema=_GeminiReview` 會強制 Gemini 傳回符合 Pydantic 模型的有效 JSON。不必使用規則運算式，也不必透過提示工程設計格式，因為模型會在 API 級別受到限制。

TODO 3 - Parse the response and return the result

找出 `# TODO 3` 註解，並替換為：

```
review = _GeminiReview.model_validate_json(response.text)
return ImageReviewResult(status="success", concept_name=concept_name,
**review.model_dump())
```

試試看：將下拉式選單切換為 **critic**，然後傳送：

```
Review this Instagram caption for an eco-friendly water bottle brand targeting millennials:
"Hydrate smarter, live greener. 💧 Our EcoFlow bottle tracks your intake, keeps your drink
cold for 24h, and never touches single-use plastic. Because what you drink from matters as
much as what you drink. #EcoFlow #HydrationGoals #SustainableLiving #ZeroWaste #HealthyHabits
- Shop link in bio."
Score it and indicate APPROVED or NEEDS_REVISION with specific feedback.
```

確認回應中包含 `**POSTS REVIEW:**`、`Status: APPROVED` (或 `NEEDS_REVISION`) 和 `**OVERALL ASSESSMENT:**`。如果這些區段存在，Critic 即可插入協調器。

測試完畢後，請按 `Ctrl+C` 停止伺服器。

8. 使用 MCP 建構專案經理代理

專案經理介紹了新概念：**MCP (Model Context Protocol)**。

開啟檔案：

```
cloudshell edit agents/project_manager/agent.py
```

這個檔案較為複雜，其中包含一個 `create_project_manager_agent()` 函式，具有兩個分支：一個不使用 Notion (僅限文字的時間軸)，另一個則使用 Notion MCP 工具集。請填寫這兩項資訊。

MCP 解決的問題

您的代理程式需要呼叫外部服務，例如在 Notion 中建立頁面。您可以編寫 Python 程式碼，直接呼叫 Notion REST API。但隨後：

- 每位開發人員都會編寫不同的包裝函式
- 您必須維護自訂整合代碼
- 除非手動說明每個端點，否則 LLM 不會知道 API 的存在

MCP 定義了標準方式，讓外部服務將功能公開為工具，供 **LLM** 自動探索及呼叫，解決了這個問題。

什麼是 MCP？

MCP (Model Context Protocol) 是由 Anthropic 發布的**開放標準**，可將 AI 代理連結至外部工具和資料來源。功能如同通用轉接器。

MCP 伺服器是小型程式，可執行下列動作：

1. 包裝外部 API (Notion、GitHub、資料庫、檔案系統...)
2. 將該 API 公開為類型化、附有說明文件的**工具清單**
3. 透過簡單的通訊協定 (stdio 或 HTTP) 與代理程式通訊

代理會連線至 MCP 伺服器，自動探索可用工具，並像呼叫其他工具一樣呼叫這些工具 - LLM 會將 `API-post-page(...)` 視為可呼叫的函式。

A2A 與 MCP 有何不同？

這一點常讓人混淆。兩者之間的主要差異如下：

	A2A	MCP
連結內容	代理人 ↔ 代理人	代理程式 ↔ 外部工具/服務
另一面是	另一個 LLM 代理程式	API 包裝函式 (不含 LLM)
範例	創意總監致電品牌策略師	專案經理呼叫 Notion API

通訊協定	透過 HTTPS 的 JSON-RPC	stdio 或 HTTP 串流
定義者	Google	Anthropic

您可以這樣理解：

- **A2A** = 代理程式與其他代理程式的通訊方式
- **MCP** = 代理與工具和服務的通訊方式

在這個專案中，這兩者會一起使用：



MCP 在這個專案中的運作方式

代理執行時，ADK 會以子程序的形式啟動 `notion-mcp-server`。這個程序會直接向 LLM 公開這些工具：

工具	用途
<code>API-retrieve-a-database</code>	擷取結構定義 (屬性名稱、類型、有效值)
<code>API-post-database-query</code>	查詢現有網頁
<code>API-post-page</code>	建立新頁面
<code>API-patch-page</code>	更新現有頁面

LLM 會像呼叫任何其他函式一樣呼叫這些函式，完全不知道這些函式會在幕後透過 MCP 傳送至 Notion REST API。

為什麼要使用 stdio？為什麼不使用 HTTP 就好？

MCP 伺服器會以代理的子程序形式執行，並透過 stdin/stdout 通訊。也就是說：

- 不需額外網路連接埠
- 生命週期由代理程式管理 (視需求啟動，在結束時停止)
- 所有內容都以一個 Docker 映像檔出貨，不必部署個別服務

(選用) 啟用 Notion 整合

你可以略過整個章節。無論是否使用 Notion，專案經理代理一律會產生完整的文字廣告活動時間表。如果略過這項設定，服務專員會改用記憶體內模式，並在對話中以純文字輸出時間軸。不會有任何問題，只是工作不會顯示在 Notion 資料庫中。如要略過，請直接前往 **TODO 1**。

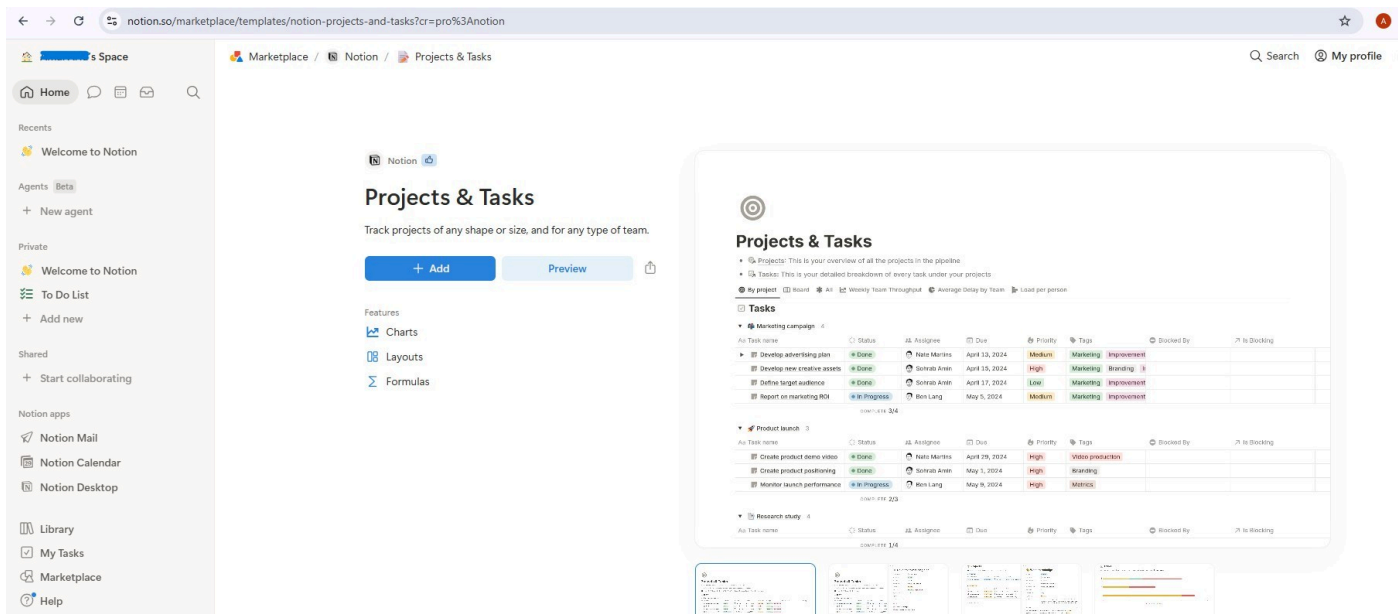
如果您有 Notion 帳戶，並想實際體驗 MCP 整合功能，請立即完成下列設定。後續的待辦事項會參照 Notion 資料庫 ID，您可以在這裡取得這些 ID。

步驟 1 - 透過範本建立 Notion 資料庫

我們使用官方的 **Notion 專案與工作** 範本做為資料庫。我們刻意選擇這個範本，是為了展示複雜的實際設定，其中包含多種屬性類型 (狀態、日期範圍、關係、選取)，且名稱並非顯而易見。這項測試非常適合用來驗證 MCP 的動態結構定義探索功能：代理程式必須在執行階段找出確切的屬性名稱，而不是將這些名稱硬式編碼。

按一下下方連結，將範本新增至 Notion 工作區：

→ 將「專案與工作」範本新增至 Notion

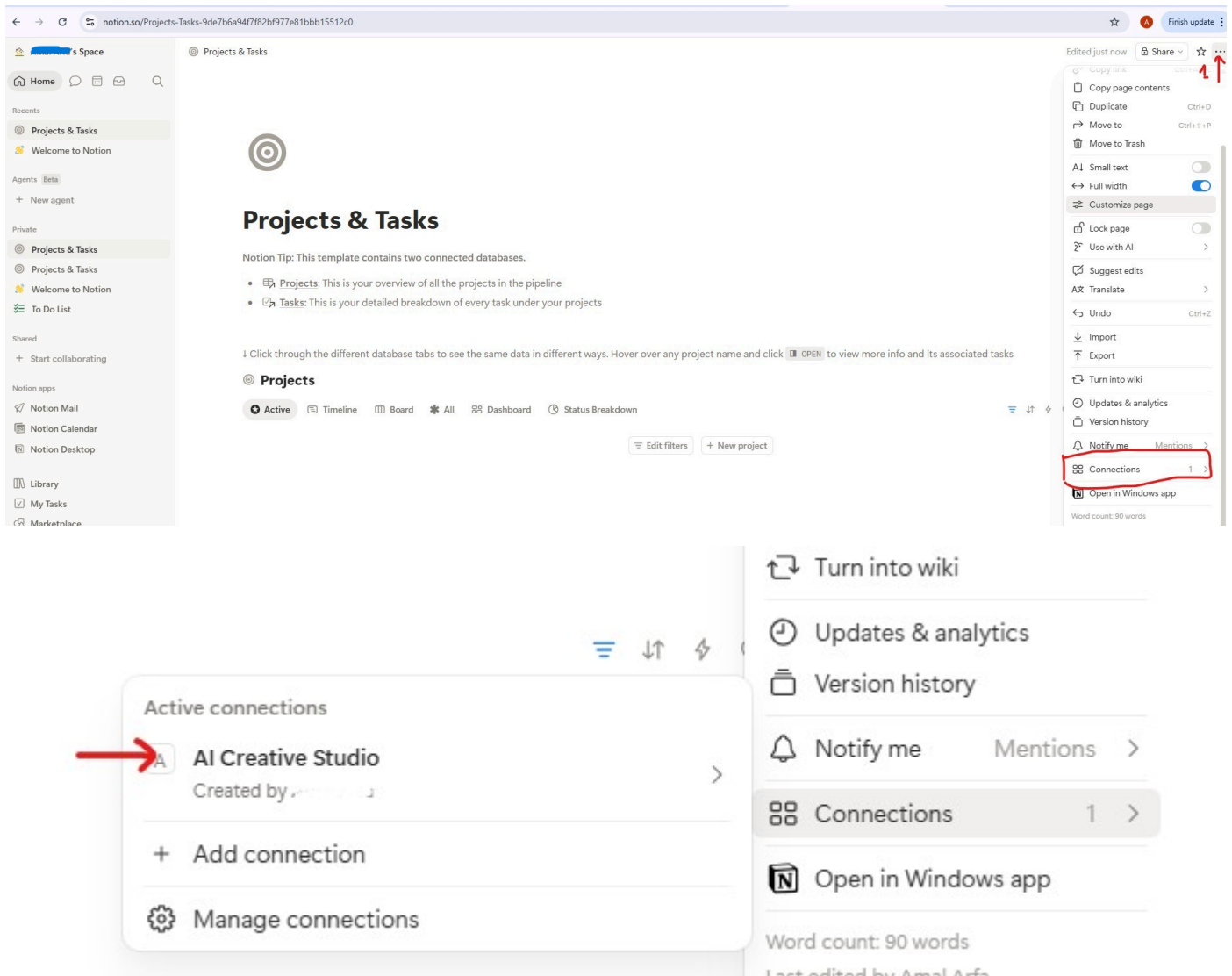


新增後，您會有兩個連結的資料庫：「專案」和「工作」。範本隨附範例項目，請**全部刪除**再繼續操作，讓服務專員從乾淨的工作區開始 (全選 → 刪除)。

步驟 2 - 建立 Notion 整合

建立整合：

1. 前往 notion.so/my-integrations
2. 按一下「New Integration」→ 將其命名為 **AI Creative Studio**
3. 與工作區建立關聯
4. 按一下「設定」→ 確保「讀取內容」、「更新內容」和「插入內容」功能都已勾選



8. 對「Tasks」資料庫執行相同操作

取得資料庫 ID：

9. 按一下「Projects」資料庫連結開啟該資料庫，系統會在新頁面中開啟資料庫，網址類似於：

<https://www.notion.so/9887b6a94f7f83f68f8581e038d1aaa4?v=2c37b6a94f7f838685f1086e312c7278>

Projects & Tasks

Notion Tip: This template contains two connected databases.

→ **Projects:** This is your overview of all the projects in the pipeline

→ **Tasks:** This is your detailed breakdown of every task under your projects

資料庫 ID 是網址中的第一個 UUID，也就是 `?v=` 前的所有內容：

```
https://www.notion.so/{DATABASE_ID}?v=...
          ^^^^^^^^^^^^^^^^^
          9887b6a94f7f83f68f8581e038d1aaa4 ← this is your DATABASE_ID
```

10. 對「Tasks」資料庫連結執行相同操作，取得資料庫 ID

11. 將這三個值都新增至 `.env`：

```
NOTION_TOKEN=ntn_your-token-here
NOTION_PROJECT_DATABASE_ID=9887b6a94f7f83f68f8581e038d1aaa4 # <-- your Projects DB ID
NOTION_TASKS_DATABASE_ID=your-tasks-db-id # <-- your Tasks DB ID
```

步驟 3 - 安裝 Notion MCP 伺服器

專案管理員會透過官方 `@notionhq/notion-mcp-server` Node.js 套件連線至 Notion。全域安裝：

```
npm install -g @notionhq/notion-mcp-server@1.9.1
```

驗證安裝狀態：

```
npm list -g @notionhq/notion-mcp-server
```

預期輸出內容：

```
└── @notionhq/notion-mcp-server@1.9.1
```

notion-mcp-server: command not found

? 確定已安裝 Node.js (`node --version`)，且 npm 全域 bin 位於 PATH (`export PATH=$PATH:$(npm bin -g)`)。

步驟 4 - 驗證 .env

開啟 `.env`，確認已設定所有三個 Notion 值 (您在步驟 2 中新增了這些值)：

```
cloudshell edit .env
```

```
NOTION_TOKEN=ntn... # integration token
NOTION_PROJECT_DATABASE_ID=... # Projects database ID
NOTION_TASKS_DATABASE_ID=... # Tasks database ID
```

Project Manager 代理會在啟動時自動偵測這些變數，並啟用 Notion MCP 工具集。

結構定義探索的運作方式

專案管理員使用動態結構定義探索功能，絕不會將 Notion 屬性名稱硬式編碼：

```
Step 1: Call API-retrieve-a-database to discover exact property names
Step 2: Read the "properties" object in the response
Step 3: Use ONLY discovered property names (case-sensitive) in API calls
Step 4: For select/status fields, use only values from the options array
```

也就是說，無論 Notion 資料庫結構為何，代理程式都會自動調整。即使將屬性重新命名為法文、阿拉伯文或其他語言，代理程式仍可正常運作。

TODO 1 - Write the system instruction

啟動器已計算 `notion_section` - Notion 未設定時為空字串，設定時則為包含資料庫 ID 和完整工具指引的區塊。這可確保 Notion 指令完全不會出現在沒有 Notion 的代理程式提示中；LLM 絕不會看到沒有的工具規則。

您的工作是將預留位置 `return` 替換為使用 `{notion_section}` 的實際系統指令：

```
return f"""You are a Project Manager specializing in creative campaign execution.

Today's date is {datetime.date.today().strftime("%B %d, %Y")}.
Use this as the starting point for all timelines.

Your goal: create a complete project plan for the campaign.
{notion_section}

**Project Timeline:**
Phase 1: Strategy & Research | [date] → [date] | [key activities]
Phase 2: Content Creation | [date] → [date] | [key activities]
Phase 3: Review & Revision | [date] → [date] | [key activities]
Phase 4: Launch & Monitoring | [date] → [date] | [key activities]

**Task List:**
| Task | Owner | Deadline | Status |
[list each task with realistic deadlines from today; set Owner to TBD]

**Budget Breakdown:**
[by category with approximate allocations]

**Milestones:**
[3-5 key checkpoints with dates]

**Notion Status:**
[What happened - e.g. "Project created (ID: xxx), 8 tasks linked" or "Notion not configured - text timeline only"]
"""
```

待辦事項 2 - 沒有 Notion 的代理程式

在 `create_project_manager_agent()` 的 `if not notion_token` 分支中，將不完整的代理程式替換為：

```
return Agent(
    name="project_manager",
    model=os.getenv("GEMINI_MODEL", "gemini-2.5-flash"),
    generate_content_config=GENERATE_CONTENT_CONFIG,
    instruction=get_system_instruction(),
    description="Project manager that creates campaign timelines and task
breakdowns",
)
```

TODO 3 - Agent with Notion MCP

注意：範例檔案已在 `create_project_manager_agent()` 上方包含預先撰寫的 `handle_notion_error` 回呼。這項功能會攔截 Notion API 錯誤 (400/404)，並以清楚明瞭的實用訊息取代原始錯誤酬載，讓 LLM 自行修正錯誤。你只需要透過 `after_tool_callback` 接線即可。

首先，請在 `create_project_manager_agent()` 頂端讀取兩個資料庫 ID：

```
notion_token          = os.getenv("NOTION_TOKEN")
notion_project_db_id  = os.getenv("NOTION_PROJECT_DATABASE_ID")
notion_tasks_db_id    = os.getenv("NOTION_TASKS_DATABASE_ID")
```

接著在 `else` 分支中，建立 MCP 工具集和代理程式：

```

from google.adk.tools.mcp_tool import McpToolset, StdioConnectionParams
from mcp import StdioServerParameters

server_params = StdioServerParameters(
    command="notion-mcp-server",
    env={
        "NOTION_TOKEN": notion_token,
        "PATH": os.environ.get("PATH", ""),
    }
)
notion_toolset = McpToolset(
    connection_params=StdioConnectionParams(
        server_params=server_params,
        timeout=30.0
    )
)

return Agent(
    name="project_manager",
    model=os.getenv("GEMINI_MODEL", "gemini-2.5-flash"),
    generate_content_config=GENERATE_CONTENT_CONFIG,
    after_tool_callback=handle_notion_error,
    instruction=get_system_instruction(
        project_database_id=notion_project_db_id,
        tasks_database_id=notion_tasks_db_id,
    ),
    description="Project manager with Notion integration for task tracking",
    tools=[notion_toolset],
)

```

最佳做法：請勿對選用整合功能進行硬性失敗。文字時間軸一律是主要交付項目，Notion 則為輔助項目。

使用 ADK Web 在本機測試專案經理

```
uv run adk web agents --allow_origins='*'
```

開啟通訊埠 8000 的網頁預覽。使用代理程式下拉式選單選取 **project_manager**，然後嘗試：

Create a project plan for a GreenBrew organic coffee brand Instagram campaign.
 Budget: \$2,500. Launch in 3 weeks. Target audience: eco-conscious millennials aged 22-30.
 Include phases, tasks with deadlines from today, and milestones.

請在此使用其他廣告活動。稍後，完整的自動化調度管理工具執行作業會使用 EcoFlow 智慧水瓶。在這裡使用 GreenBrew，可在檢查 Notion 或查看輸出內容時區分這兩項測試。

畫面上會顯示結構化文字時間軸，包含階段、工作清單和里程碑。如果 `.env` 中已設定 Notion 憑證，代理程式也會在 Notion 工作區中建立項目。

9. 瞭解 A2A 通訊協定

我們會使用 Agent2Agent 通訊協定 (A2A) 連結系統中的不同代理。讓我們瞭解這項功能的運作方式。

A2A 解決的問題

假設您有一個使用 ADK 建構的品牌策略師代理，以及一個使用 LangGraph 建構的文案人員代理。如何撥打電話給對方？它們使用不同的內部語言。每次都必須撰寫自訂黏合程式碼。

A2A 定義了通用語言，任何代理 (無論架構為何) 都能使用，因此可解決這個問題。這是代理程式世界的 HTTP：大家一致同意的標準，因此任何人都能與任何人對話。

什麼是 A2A？

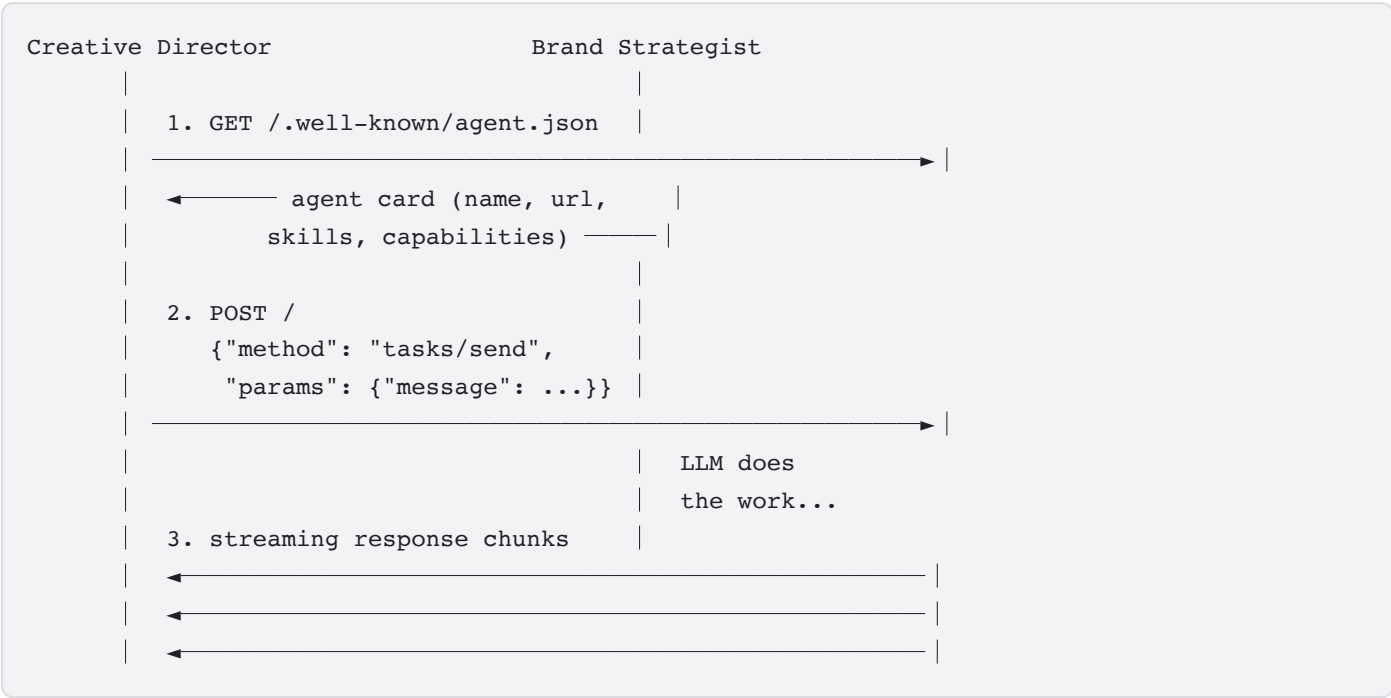
Agent-to-Agent (A2A) 是 Google 發布的代理通訊開放標準。定義：

- 1. 代理如何描述自己 - `/.well-known/agent.json` 的代理資訊卡
- 2. 其他代理程式如何呼叫這個代理程式 - 透過 HTTPS 的 JSON-RPC
- 3. 傳回結果的方式 - 串流或單一回應

A2A 的彈性：

- 不限語言 - Python 代理程式可以與 TypeScript 代理程式對話
- 不限框架 - ADK 代理可與 LangGraph 或 CrewAI 代理對話
- 不限基礎架構：本機代理程式可以與雲端代理程式通訊

運作方式 - 逐步說明



步驟 1 - 探索：協調器會擷取代理資訊卡一次，瞭解代理的名稱、網址和功能。

步驟 2 - 呼叫：協調器會透過 JSON-RPC POST 傳送工作。內文包含訊息 (專家的提示)。

步驟 3 - 回覆：專員會以區塊形式串流回覆，就像一般的 LLM 呼叫。

服務專員資訊卡

每個代理都會在 `/.well-known/agent.json` 發布自我描述。這就像商家名片，可向全世界說明代理程式的功能和聯絡方式：

```
{
  "name": "brand_strategist",
  "description": "Market research and competitive analysis",
  "url": "https://brand-strategist-xyz.run.app",
  "capabilities": { "streaming": true },
  "skills": [
    {
      "id": "market_research",
      "description": "Research target audiences, competitors, and trends"
    }
  ]
}
```

協調器會讀取這張卡片來建構 `RemoteA2aAgent` 物件，不需要專家內部結構的硬式編碼知識。

在 ADK 中透過 A2A 公開代理

`to_a2a()` 會將任何 ADK 代理包裝在符合 A2A 規範的 FastAPI 應用程式中。一程式碼：

```
from google.adk.a2a.utils.agent_to_a2a import to_a2a

# root_agent = your normal ADK Agent(...)
a2a_app = to_a2a(root_agent, host=PUBLIC_HOST, port=PUBLIC_PORT, protocol=PROTOCOL)
uvicorn.run(a2a_app, host=HOST, port=PORT)
```

系統會自動建立：

- `/.well-known/agent.json` - 代理資訊卡
 - `/` - JSON-RPC 端點 (所有 A2A 工作要求都會前往根路徑)
-

10. 將代理公開為 A2A 服務

如要將代理程式公開為 A2A 服務，可以使用 ADK 的 `to_a2a()` 公用函式。

`to_a2a()` 的運作方式

```
from google.adk.a2a.utils.agent_to_a2a import to_a2a

a2a_app = to_a2a(root_agent, host=PUBLIC_HOST, port=PUBLIC_PORT, protocol=PROTOCOL)
uvicorn.run(a2a_app, host=HOST, port=PORT)
```

`to_a2a()` 會將 ADK 代理封裝在 FastAPI 應用程式中，自動公開：

- `/.well-known/agent.json` - 代理程式資訊卡 (名稱、說明、功能)
- `/a2a/{agent_name}` - 用於接收工作的 JSON-RPC 端點

每個代理程式的架構程式碼都已包含 `__main__` 區塊，可使用 `to_a2a()` 將代理程式包裝在 A2A 伺服器中。您不需要編寫這段程式碼，因為我們已提供。

瞭解雙網址設定

執行 `python agent.py` 時，`__main__` 區塊會使用兩個不同的網址設定：

```
# Where the server actually listens (network interface):
HOST = "0.0.0.0"
PORT = 8082 # Brand Strategist (others use 8083–8086 locally)

# What gets advertised in the agent card (the address other agents use to reach it):
PUBLIC_HOST = os.getenv("PUBLIC_HOST", "localhost")
PUBLIC_PORT = int(os.getenv("PUBLIC_PORT", str(PORT)))
PROTOCOL     = os.getenv("PROTOCOL", "http")

a2a_app = to_a2a(root_agent, host=PUBLIC_HOST, port=PUBLIC_PORT, protocol=PROTOCOL)
uvicorn.run(a2a_app, host=HOST, port=PORT)
```

環境	<code>HOST:PORT</code> (聆聽次數)	<code>PUBLIC_HOST:PUBLIC_PORT</code> (在代理商資訊卡中宣傳)
局部	<code>0.0.0.0:8082</code>	<code>http://localhost:8082</code>
Cloud Run	<code>0.0.0.0:8080</code>	<code>https://brand-strategist-xyz.run.app:443</code>

在本地，兩者都指向同一部機器。在 Cloud Run 上，容器會在內部監聽 `8080`，但代理程式資訊卡必須通告公開 HTTPS 網址，否則創意總監無法從容器外部連線至專員。

啟動所有 5 部專用 A2A 伺服器

讓我們同時以 A2A 伺服器身分執行所有 5 位專家，然後在本機測試指向他們的創意總監。

開啟 **5 個不同的 Cloud Shell 終端機** (點選終端機分頁列中的 **+** 圖示)，並在每個終端機中執行一個代理程式。

`uv run` 會自動啟用 `.venv`，因此每個終端機都不需要手動 `source`。

終端 1 - 品牌策略師 (通訊埠 8082)：

```
cd ~/ai-creative-studio/workshop/starter
PORT=8082 uv run agents/brand_strategist/agent.py
```

終端 2 - 文案人員 (通訊埠 8083)：

```
cd ~/ai-creative-studio/workshop/starter
PORT=8083 uv run agents/copywriter/agent.py
```

終端 3 - 設計師 (通訊埠 8084)：

```
cd ~/ai-creative-studio/workshop/starter
PORT=8084 uv run agents/designer/agent.py
```

Terminal 4 - Critic (port 8085)：

```
cd ~/ai-creative-studio/workshop/starter
PORT=8085 uv run agents/critic/agent.py
```

終端 5 - 專案經理 (通訊埠 8086)：

```
cd ~/ai-creative-studio/workshop/starter
PORT=8086 uv run agents/project_manager/agent.py
```

在 `.env` 中設定 `localhost` 網址

在「Terminal 6」中，使用本機代理程式網址更新 `.env`，讓創意總監可以找到這些網址：

```
cd ~/ai-creative-studio/workshop/starter

sed -i \
-e 's|STRATEGIST_AGENT_URL=.*|STRATEGIST_AGENT_URL=http://localhost:8082|' \
-e 's|COPYWRITER_AGENT_URL=.*|COPYWRITER_AGENT_URL=http://localhost:8083|' \
-e 's|DESIGNER_AGENT_URL=.*|DESIGNER_AGENT_URL=http://localhost:8084|' \
-e 's|CRITIC_AGENT_URL=.*|CRITIC_AGENT_URL=http://localhost:8085|' \
-e 's|PM_AGENT_URL=.*|PM_AGENT_URL=http://localhost:8086|' \
.env
```

使用 A2A 檢查器檢查代理

A2A 檢查器是開放原始碼開發人員工具，可原生支援 A2A 通訊協定。您可以直接連線至任何正在執行的 A2A 代理、讀取代理資訊卡，以及傳送工作，完全不需要編寫任何用戶端程式碼。

顯示內容：

- **代理程式資訊卡**：代理程式宣傳的結構化中繼資料，包括名稱、說明、支援的輸入/輸出模式和端點網址。這是 Creative Director 發現專員時讀取的內容。
- **對話介面**：透過 A2A 將任何訊息傳送給代理程式，並查看原始回覆。您可以先單獨測試提示詞，再將代理程式連結在一起。
- **通訊協定驗證**：檢查員會檢查代理程式資訊卡是否符合 A2A 規格，並提早顯示缺少的欄位或格式錯誤的回覆。

重要性：稍後部署至 Cloud Run 時，Creative Director 會從 `/well-known/agent.json` 擷取各專家的代理程式資訊卡，如果該資訊卡有誤（網址錯誤或缺少功能），自動化調度管理工具會無聲無息地失敗。檢查器可讓您在本地找出這些問題，再部署至雲端。

▼ Agent Card

Agent card is valid.

```
{
  "capabilities": {},
  "defaultInputModes": [
    "text/plain"
  ],
  "defaultOutputModes": [
    "text/plain"
  ],
  "description": "Brand strategist for market research, trend analysis, and competitive insights",
  "name": "brand_strategist",
  "preferredTransport": "JSONRPC",
  "protocolVersion": "0.3.0",
  "skills": [
    {
      "description": "Brand strategist for market research, trend analysis, and competitive insights I am a Brand Strategist specializing in market research and trend analysis.\n\nIMPORTANT: Today's date is April 08, 2026.\nWhen conducting research, focus on current trends from 2026.\nUse search queries like \"[topic] trends 2026\" for recent insights.\n\nIMPORTANT: my role is RESEARCH ONLY. I do NOT create campaign content, captions, or designs.\nAfter providing research insights, my work is complete.\n\nmy expertise:\n- Identifying target audience insights and behaviors\n- Analyzing competitor strategies\n- Researching current social media trends\n- Understanding platform algorithms and best practices\n\nI have access to:\n- google_search: Search the web for competitors, trends, and market insights\n\nWhen given a campaign brief:\n1. Use google_search to research the target audience's current interests\n2. Search for and analyze 2-3 competitor brands\n3. Identify 3-5 trending topics related to the product category\n4. Provide high-level strategic insights - NOT specific campaign content\n\nDO NOT create captions, copy, designs, or any campaign content.\n\nFormat my output as:\n**Audience Insights:**\n[Key behaviors and preferences based on research]\n\n**Competitive Analysis:**\n[What 2-3 competitors are doing - strengths and weaknesses]\n\n**Trending Topics:**\n[3-5 relevant trends to consider]\n\n**Key Strategic Insights:**\n[High-level themes and positioning opportunities]\n",
      "name": "brand_strategist"
    }
  ]
}
```

代理資訊卡會顯示專家的身分和能力，與其他代理看到的資訊完全相同。

▼ Agent Card

Agent card is valid.

```
researching current social media trends (in understanding platform algorithms and best practices (in) have access to:\n
google_search: Search the web for competitors, trends, and market insights\n\nWhen given a campaign brief:\n1. Use
google_search to research the target audience's current interests\n2. Search for and analyze 2-3 competitor brands\n3. Identify
3-5 trending topics related to the product category\n4. Provide high-level strategic insights – NOT specific campaign
content\n\nDO NOT create captions, copy, designs, or any campaign content.\n\nFormat my output as:\n**Audience
Insights:**\n[Key behaviors and preferences based on research]\n\n**Competitive Analysis:**\n[What 2-3 competitors are doing –
strengths and weaknesses]\n\n**Trending Topics:**\n[3-5 relevant trends to consider]\n\n**Key Strategic Insights:**\n[High-
level themes and positioning opportunities]\n",
  "id": "brand_strategist",
  "name": "model",
  "tags": [
    "llm"
  ]
},
{
  "description": "google_search",
  "id": "brand_strategist-google_search",
  "name": "google_search",
  "tags": [
    "llm",
    "tools"
  ]
}
],
"supportsAuthenticatedExtendedCard": false,
"url": "https://8082-cs-682664025072-default.cs-europe-west1-iuzs.cloudshell.dev",
"version": "0.0.1"
}
```

安裝並啟動檢查器

Cloud Shell 磁碟空間：安裝 A2A Inspector 需要約 500 MB。Cloud Shell 的主目錄有 5 GB 的限制，所有專案共用這個空間。如果遇到「空間不足」錯誤，請先釋出空間：

```
df -h ~
pip cache purge
rm -rf ~/.cache/uv
```

```
cd ~/ai-creative-studio/workshop
./setup_inspector.sh
```

下載逾時？Cloud Shell 設有間歇性的連出頻寬限制。請使用較長的逾時時間重試：

```
UV_HTTP_TIMEOUT=120 ./setup_inspector.sh
```

找不到模組「dompurify」的宣告檔案：如果收到這則錯誤訊息，表示您可能先前已安裝 A2A 檢查工具。刪除目錄並重試：

```
rm -rf ~/a2a-inspector/
```

如果仍有問題，可以略過使用 A2A 檢查工具進行測試，繼續完成其餘程式碼研究室活動。

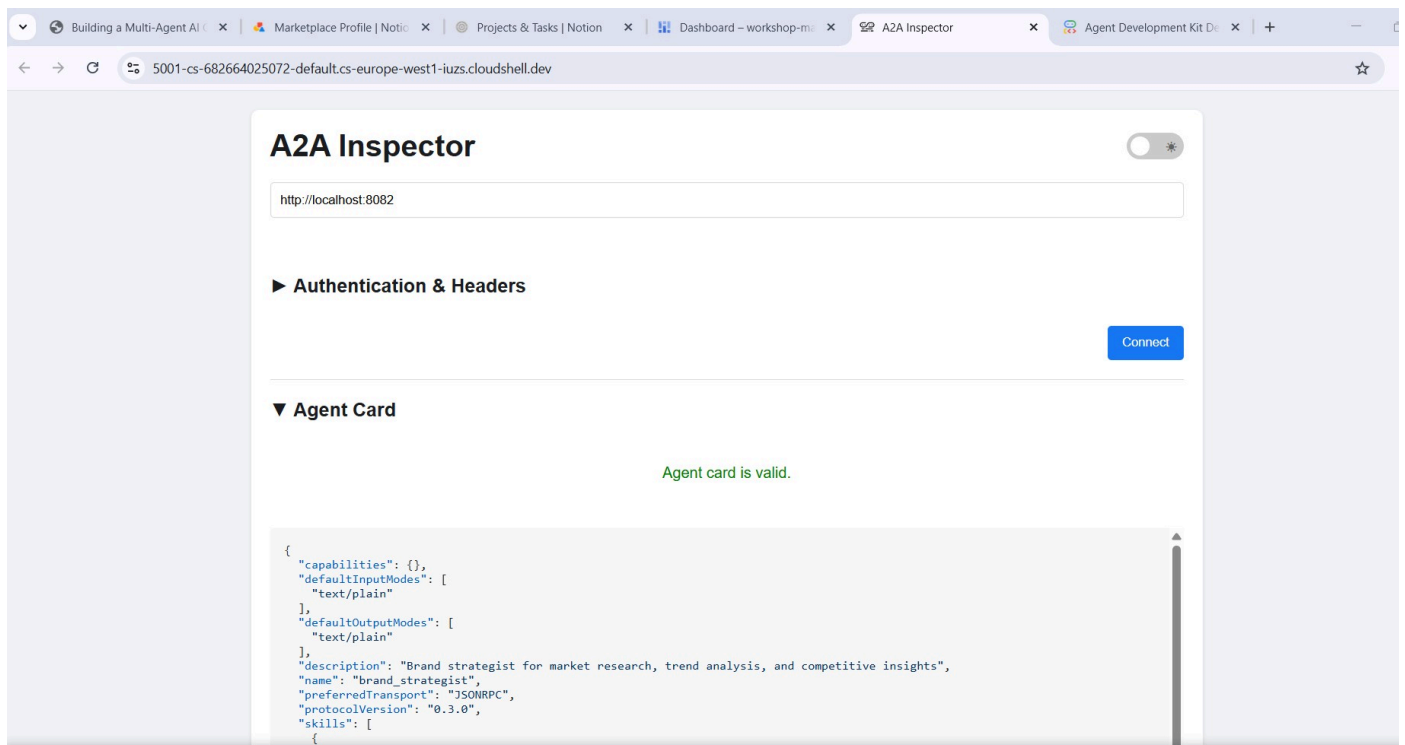
`.env` 更新是單次指令。使用「Terminal 6」啟動檢查器：

```
cd ~/a2a-inspector
bash scripts/run.sh
```

如要開啟檢查器 UI，請依序點選「Web Preview」(網頁預覽) → 「Change port」(變更通訊埠) → 輸入 5001。

與品牌策略專家聯絡

在檢查器的網址欄位中輸入 `http://localhost:8082`，然後按一下「連線」。檢查員會擷取服務專員卡片，並顯示專家的中繼資料。



代理程式資訊卡提供的資訊

代理商資訊卡不只是中繼資料，更是代理商向電視網宣傳的完整功能合約。連線至專案管理員 (`http://localhost:8086`)，查看最豐富的範例：

```
{
  "name": "project_manager",
  "description": "Project manager with Notion integration for task tracking",
  "protocolVersion": "0.3.0",
  "defaultInputModes": ["text/plain"],
  "defaultOutputModes": ["text/plain"],
  "skills": [
    {
      "id": "project_manager",
      "name": "model",
      "tags": ["llm"],
      "description": "... full system instruction including today's date and Notion database
IDs ..."
    },
    {
      "id": "project_manager-API-post-page",
      "name": "API-post-page",
      "tags": ["llm", "tools"],
      "description": "Notion | Create a page"
    },
    {
      "id": "project_manager-API-retrieve-a-database",
      "name": "API-retrieve-a-database",
      "tags": ["llm", "tools"],
      "description": "Notion | Retrieve a database"
    }
  ]
}
```

主要有以下三點：

- 1. MCP 工具會變成 A2A 技能** - 專案經理有權存取的所有 Notion 工具 (`API-post-page` 、 `API-retrieve-a-database` 等) 都會列為代理程式資訊卡中的個別技能。網路上的任何其他代理程式都可以探索這個代理程式可使用的工具，不必讀取任何程式碼。
- 2. 系統指令已嵌入** - 第一項技能的 `description` 包含完整的系統指令，包括今天的日期和 Notion 資料庫 ID。創意總監會透過這種方式，瞭解在呼叫專案經理時要傳遞的內容。
- 3. 網址是即時端點** - `url` 欄位是創意總監呼叫這位專家時， `RemoteA2aAgent` 實際使用的內容。如果資訊卡中的網址有誤，協調器就無法連上代理程式。

這就是檢查器成為強大偵錯工具的原因：只要看一眼代理程式資訊卡，就能知道代理程式是否正在執行、有哪些工具，以及端點是否正確。

傳送測試訊息

連線後，在對話面板中輸入提示並傳送。檢查員會將其提交為 A2A 工作，並將回應串流回傳，這與創意總監在製作環境中呼叫這個代理程式的方式相同。

5001-cs-682664025072-default.cs-europe-west1-iuzs.cloudshell.dev

Chat

New Session

Messages from the agent are marked with ✔ (compliant) or ⚠ (non-compliant). Click any message to view the raw JSON.

▶ Session Details

What are the top Instagram trends in the wellness space in 2026?

task

Audience Insights: Gen Z is a significant driver of wellness trends in 2026, with 60% prioritizing health above career or money. Social media plays a substantial role, influencing the wellness interests of one in three younger consumers. This demographic values authenticity, community, and "digital well-being resets," indicating a pushback against constant tech use. They are moving away from fad dieting towards purposeful and mindful nutrition and consider skincare and sun protection as non-negotiable aspects of their routine. Gen Z also favors small, consistent habits and social wellness experiences.

Competitive Analysis: Successful brands in the 2026 wellness space are focusing on clarity, credibility, and connection. They are utilizing integrated communication strategies that combine scientific backing with compelling storytelling, and leveraging both experts and influencers. There's a strong emphasis on hyper-personalized wellness, moving beyond general fitness apps

▶ Message Metadata

Type a message...

+Send

將檢查器指向任一本地通訊埠 (8082 至 8086)，即可個別測試每位專家。

步驟 11 · 約 15 分鐘

11. 建構創意總監自動調度管理工具

創意總監是主調度員，這項工具會從環境變數讀取專家網址，將每個網址包裝為 `RemoteA2aAgent`，並以 `AgentTool` 的形式公開，供 LLM 呼叫。

確認 5 個專員代理程式仍在執行中 (步驟 10 中的終端機 1 至 5)。

在「Terminal 6」(A2A 檢查器終端機) 中，使用 `Ctrl+C` 停止檢查器。

開啟檔案：

```
cd ~/ai-creative-studio/workshop/starter
cloudshell edit agents/creative_director/agent.py
```

這個檔案有三個 TODO。請依序完成這些步驟。

TODO 1 - Review the already written system instruction

系統指令位於同一目錄的 `prompt.py` 中，系統會自動匯入：

```
from .prompt import SYSTEM_INSTRUCTION_TEMPLATE
```

請先開啟 `prompt.py` 閱讀相關資訊，再繼續操作：

```
cloudshell edit agents/creative_director/prompt.py
```

瞭解這項設定非常重要，因為它會控管整個自動化調度管理行為。

為何協調器提示會控制一切

請開啟 `prompt.py`，並參考下方範例中特定部分。

`prompt.py` 中的提示不只是文件，更是整個系統的控制層。如果自動調度管理工具提示結構不良，就會導致：代理的呼叫順序錯誤、內容是由自動調度管理工具而非專家生成、工作流程在失敗後繼續執行，以及代理之間默默捨棄脈絡。這九項元素可避免最常見的失敗情況：

要素 0 - 先規劃，再執行

這是最重要的元素。在呼叫任何專家前，系統會指示協調器輸出編號計畫：

```
I'll create your campaign by coordinating the specialist agents in sequence:
1. Brand Strategist - develop positioning and audience insights
2. Copywriter - write captions using those insights
3. Visual Designer - create image prompts aligned with the copy
4. Critic - review and score the full package
5. Project Manager - build the timeline and task breakdown
```

如果沒有這個步驟，LLM 會直接跳到工具呼叫，並在工作流程中迷失方向，尤其是在收到專家的長篇回覆後。先列出計畫大綱，有助於自動化調度管理工具掌握進度、後續步驟，以及完整執行流程。如果略過這項步驟，自動調度管理工具可能會在工作流程中途停止運作，或重複執行步驟。

要素 1 - 明確的角色定義

- ✗ "You are a helpful creative assistant."
- ✓ "You orchestrate specialists. You do NOT write captions, designs, or timelines yourself."

如果沒有明確禁止，LLM 有時會略過呼叫專家，直接生成內容，因為這樣比較快，而且 LLM 「知道」該怎麼做。這項指令必須讓這個結果錯誤。

元素 2 - 含有錯誤模式的工具呼叫語法

只顯示正確語法是不夠的，大型語言模型可能會生成看似合理的呼叫，但會無聲失敗。提示會明確列出正確的模式，以及絕對不能使用的模式：

- ✓ `copywriter(request="...")` ← correct
- ✗ `print(copywriter(...))` ← breaks silently
- ✗ `default_api.copywriter(...)` ← breaks silently
- ✗ `copywriter.run(...)` ← breaks silently
- ✗ `agents.copywriter(...)` ← breaks silently

明確列出錯誤模式後，正式環境中格式錯誤的工具呼叫減少了約 95%。

元素 3 - 逐步說明循序執行作業

- a) Call the tool
- b) Wait for tool_output
- c) Verify the output is not an error
- d) Confirm to the user: "✓ Brand Strategist complete"
- e) Then move to the next agent

如果沒有步驟 (b) 和 (c)，大型語言模型有時會同時呼叫兩個代理程式，或是在收到回應前就假設成功並繼續執行。

元素 4 - 錯誤指令：停止、回報、請勿繼續

在早期版本中，協調器會收到某位專家的錯誤訊息，並為該訊息捏造合理的輸出內容，然後繼續與下一個專員互動。使用者以虛構的基礎建立廣告活動，但看起來很完整。修正方式很明確：**立即停止**。**回報確切的錯誤**。**請勿繼續行駛**。

元素 5 - 內容傳遞規則

遠端代理程式**不會保留對話記錄**。當自動調度管理工具透過 A2A 呼叫 Copywriter 時，Copywriter 只會看到該單一要求中的訊息，完全不知道 Brand Strategist 說了什麼。自動調度管理工具必須明確將先前的輸出內容併入後續每次呼叫：

```
copywriter(request="Create 3 posts for EcoFlow water bottle targeting millennials.
Use these insights from the Brand Strategist: [paste full strategist output here].
Create engaging captions with hashtags.")
```

說明中明確指出：「遠端代理程式『沒有』共用記憶體，您必須明確傳遞先前的輸出內容。」否則每個代理都會盲目運作。

元素 6 - 要求分類：簡單與複雜

並非每項要求都需要所有五位代理人。提示會指示協調器在規劃前對要求進行分類：

```
SIMPLE → one agent needed
"Research the eco-friendly water bottle market" → brand_strategist only
"Write 3 Instagram captions" → copywriter only

COMPLEX → all agents sequentially
"Create a complete campaign with timeline" → all 5 agents
```

如果沒有這項分類，協調器會為每項要求 (包括「請提供 3 個貼文構想」) 執行所有五個代理程式，增加不必要的延遲時間和成本。

要素 7 - 溝通規則：顯示完整輸出內容，不進行篩選

提示明確指出，協調器不得摘要或編輯專家傳回的內容：

- DO NOT summarize unless the output exceeds 2000 words
- DO NOT filter or edit agent responses
- Show the user exactly what each specialist produced
- NEVER say results are ready unless you received them in tool_output

如果沒有這項功能，協調器會用自己的話改寫專家輸出內容，導致細節遺失、產生錯誤，並失去聘請專家的意義。

要素 8 - 工作流程完成：絕不提早停止

這是一種細微但嚴重的失敗模式：協調器宣布 5 步驟計畫，完成 3 個步驟，然後呈現結果，彷彿已完成所有步驟。提示會提供明確的檢查清單，必須通過檢查，協調器才能完成作業，避免發生這種情況：

- ✓ Did I announce a plan with N agents?
 - ✓ Have I called ALL N agents from my plan?
 - ✓ Did each agent respond successfully?
 - ✓ Am I presenting complete results from ALL agents?
- If any answer is NO → continue executing the remaining agents.

這樣一來，自動調度管理系統就不會將部分執行作業視為完成。

品質驗證 (QC) 迴圈

修訂工作流程是 `prompt.py` 最複雜的部分。開啟「`## REVISION WORKFLOW`」部分，然後按照指示操作。

運作方式

評論家回覆後，創意總監不會盲目地繼續向專案經理回報。讀取評論家的輸出內容並分支：

```

Critic output
├── "All Approved: YES"
│   └── proceed to Project Manager
└── "Status: NEEDS_REVISION"
    ├── posts fail → call copywriter again with feedback
    ├── visuals fail → call designer again with feedback
    └── both fail → call copywriter, then designer
        └── revised output → Project Manager
            (1 revision max per deliverable)

```

這是 LLM 驅動，而非程式碼驅動

本程式碼研究室稍早提到，協調器會「剖析」評論家的回應。沒有 Python 程式碼會執行這項剖析作業，也沒有規則運算式或字串比對。創意總監是讀取自身指令的 LLM，該指令如下：

```

Look for "Status: NEEDS_REVISION" in the critic's response.
Posts need revision → call copywriter
Visuals need revision → call designer

```

LLM 會讀取 Critic 輸出內容中的確切字串，並按照分支執行。因此，評論家格式不得變更：如果評論家寫的是「需要修改」，而不是 `NEEDS_REVISION`，LLM 就會在指令中找不到相符項目，並略過修訂步驟。

如何在修訂呼叫中轉送情境

修訂呼叫會遵循元素 5 的相同內容傳遞規則，也就是自動調度管理工具必須明確納入所有內容，因為文案人員不會記得第一個版本：

```

"I need you to revise the Instagram posts based on critic feedback.

ORIGINAL BRIEF:
[the original user request]

YOUR FIRST VERSION:
[the posts the copywriter created]

CRITIC FEEDBACK (Score: 6/10 - NEEDS_REVISION):
[the critic's specific suggestions]

Please revise the posts addressing this feedback while maintaining
the strengths the critic identified."

```

如果沒有「YOUR FIRST VERSION」部分，文案撰寫者就會從頭開始撰寫，而不是改進已產生的內容。

只能修訂一次的原因

無論分數如何，自動化調度管理工具都會在完成一輪修訂後，將檔案轉交給專案經理。這項指令會在心理上追蹤這項資訊：

```
After calling copywriter for revision once:  
→ mark "copywriter_revised = true" in context  
→ even if the critic still suggests changes, proceed to PM
```

如果沒有這項限制，迴圈可能會無限期執行：評論家標記問題 → 文案人員修訂 → 評論家再次標記 → 文案人員再次修訂。每輪對話都會消耗權杖和時間。一次修訂就足以提升品質，不會有週期失控的風險。

傳送給專案經理的內容

專案經理一律會收到最終核准版本，而非原始版本。如果經過修訂，自動化調度管理工具會傳送修訂後的副本和圖像。如果所有項目在第一次審查時都獲得核准，就會直接通過審查。PM 不會看到遭拒的草稿。

TODO 2 - Register each specialist as a RemoteA2aAgent + AgentTool

找出 # TODO 2: For each specialist URL... 註解，並替換為：

```

if strategist_url:
    available_agents_list.append(
        "- **brand_strategist**: Market research, competitor analysis, trend
identification"
    )
    strategist_agent = RemoteA2aAgent(
        name="brand_strategist",
        description="Researches markets, competitors, and trends using Google Search",
        agent_card=f"{strategist_url}/.well-known/agent.json",
    )
    agent_tools.append(AgentTool(agent=strategist_agent))

if copywriter_url:
    available_agents_list.append(
        "- **copywriter**: Instagram captions, hashtags, and CTAs"
    )
    copywriter_agent = RemoteA2aAgent(
        name="copywriter",
        description="Creates Instagram captions with hashtags and CTAs",
        agent_card=f"{copywriter_url}/.well-known/agent.json",
    )
    agent_tools.append(AgentTool(agent=copywriter_agent))

if designer_url:
    available_agents_list.append(
        "- **designer**: Visual concepts and real images generated via Gemini (GCS URIs
returned)"
    )
    designer_agent = RemoteA2aAgent(
        name="designer",
        description="Creates visual concepts and generates real images via Gemini, stored
in GCS",
        agent_card=f"{designer_url}/.well-known/agent.json",
    )
    agent_tools.append(AgentTool(agent=designer_agent))

if critic_url:
    available_agents_list.append(
        "- **critic**: Quality review with APPROVED/NEEDS_REVISION scoring"
    )
    critic_agent = RemoteA2aAgent(
        name="critic",
        description="Reviews campaign materials and returns structured quality feedback",
        agent_card=f"{critic_url}/.well-known/agent.json",
    )
    agent_tools.append(AgentTool(agent=critic_agent))

if pm_url:
    available_agents_list.append(
        "- **project_manager**: Project timelines, task breakdowns, Notion integration"
    )
    pm_agent = RemoteA2aAgent(

```

```

        name="project_manager",
        description="Creates project timelines and task breakdowns, optionally in
Notion",
        agent_card=f"{pm_url}/.well-known/agent.json",
    )
    agent_tools.append(AgentTool(agent=pm_agent))

```

待辦事項 3 - 使用內容壓縮功能將內容包裝在應用程式中

為何需要壓縮

對話中的每則訊息 (使用者的提示、每次工具呼叫、每次工具回覆) 都會附加至 LLM 在下一個回合讀取的脈絡視窗。在 5 個代理程式的工作流程中，這項費用很快就會累積：

Turn 1:	user prompt	~200 tokens	
Turn 2:	orchestrator plan	~300 tokens	
Turn 3:	brand_strategist tool_call	~150 tokens	
Turn 4:	brand_strategist tool_output	~1,500 tokens	← full research report
Turn 5:	copywriter tool_call	~300 tokens	← must include strategist output
Turn 6:	copywriter tool_output	~2,000 tokens	← 3 captions
Turn 7:	designer tool_call	~500 tokens	
Turn 8:	designer tool_output	~1,500 tokens	
...			

到了 Agent 4 (評論家)，內容視窗會包含前三位代理的完整輸出內容，光是工具回覆就經常有 8,000 到 12,000 個權杖。即使 Gemini 2.5 Pro 的脈絡窗口很大，但隨著歷史記錄不斷增加，協調器的推理品質也會下降，因為協調器必須處理越來越多的歷史記錄。如果沒有壓縮，長時間的工作流程會達到 Agent 4 的實際限制。

壓實作業的用途

ADK 不會完整保留每個事件，而是定期呼叫 LLM，將較舊的事件摘要成精簡的表示法。系統只會在情境中保留過往事件的摘要，以及最近一次代理程式的完整輸出內容。

```

Without compaction:
    [full strategist output] + [full copywriter output] + [full designer output] + → Critic

With compaction (interval=3, overlap=1):
    [summary of strategist + copywriter] + [full designer output] + → Critic

```

摘要會保留重要事實 (重要洞察、核准的說明文字、視覺概念)，並捨棄冗長的格式、傳遞給每個代理程式的重複內容，以及中間的推理過程。Critic 仍可評估所有必要資訊，只是讀取的不是三份完整報告，而是摘要。

代碼

找出 `# TODO 3: Wrap the agent in an App...` 註解，並將預留位置 `App(...)` 替換為：

```

from google.adk.apps import App
from google.adk.apps.app import EventsCompactionConfig
from google.adk.apps.llm_event_summarizer import LlmEventSummarizer
from google.adk.models import Gemini

compaction_config = EventsCompactionConfig(
    summarizer=LlmEventSummarizer(llm=Gemini(model_id=os.getenv("GEMINI_MODEL", "gemini-2.5-flash"))),
    compaction_interval=3,    # Summarize after every 3 agent completions
    overlap_size=1,          # Keep the most recent agent's output in full
)

app = App(
    name="creative_director",
    root_agent=agent,
    events_compaction_config=compaction_config,
    plugins=[LoggingPlugin()],
)

return agent, app

```

compaction_interval=3 - 每完成 3 個代理程式後，就會觸發壓縮作業。以 5 個代理程式的管道為例，這表示管道會觸發一次 (在代理程式 1 至 3 之後)，然後查核人員和 PM 會看到代理程式 1 至 3 的摘要，以及先前代理程式的完整輸出內容。

overlap_size=1 - 系統一律會保留代理程式的完整輸出內容，絕不會摘要。這點很重要，因為評論家需要設計師的完整輸出內容 (包括 `gcs_uri` 值)，才能載入及審查實際圖片。摘要會失去這些 URI。

完整廣告活動的運作方式：

```

Agent 1 (Strategist) → full context
Agent 2 (Copywriter) → full context
Agent 3 (Designer)   → full context
                        ↓ compaction fires: summarizes agents 1-2, keeps 3 in full
Agent 4 (Critic)     → sees [summary of 1-2] + [full output of 3]
Agent 5 (PM)         → sees [summary of 1-3] + [full output of 4]

```

經驗法則：將 `compaction_interval` 設為大約一半的代理程式總數。除非下一個代理程式需要多個前任代理程式的完整詳細資料，否則請設定 `overlap_size=1`。

瞭解 RemoteA2aAgent 和 AgentTool

```
RemoteA2aAgent("brand_strategist", agent_card=url)
|
| wraps the remote service so ADK can call it
▼
AgentTool(agent=strategist_agent)
|
| exposes it as a callable tool to the LLM
▼
Agent(tools=[...])
|
| LLM calls tool("brand_strategist", message=...) when needed
▼
brand-strategist-xxxx.run.app ← actual HTTP A2A call happens here
```

LLM 會根據系統指令和使用者要求，決定*何時*呼叫各項工具。自動調度管理工具絕不會在程式碼中直接呼叫代理，一切都由 LLM 的推理能力驅動。

在本機測試 Creative Director

```
uv run adk web agents --allow_origins='*'
```

開啟通訊埠 8000 的網頁預覽。使用代理程式下拉式選單選取 **creative_director**，然後嘗試：

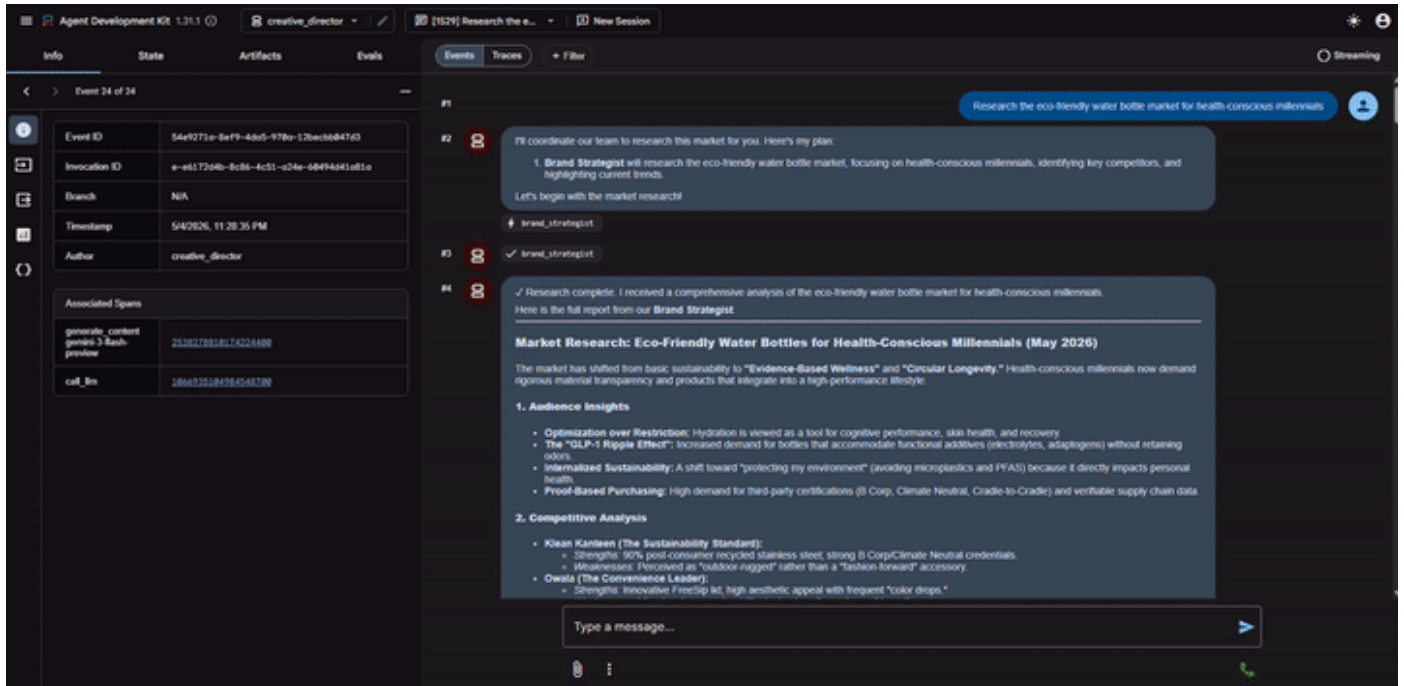
```
Research the eco-friendly water bottle market for health-conscious millennials
```

你會看到創意總監只會將這項工作轉交給品牌策略師，而你會收到品牌策略師的回覆。

如要查看完整廣告活動，請嘗試下列做法：

```
Create a complete Instagram campaign for SolarPack portable solar charger targeting
outdoor enthusiasts and digital nomads aged 22-35.
Budget $2,000, launch in 2 weeks.
```

您會看到創意總監依序協調所有 5 位專家，每位專員的輸出內容都會傳送給下一位。



service account info is missing 'email' field? 您的應用程式預設憑證已過期 (Cloud Shell 工作階段會在約 12 小時後重設)。執行 `gcloud auth application-default login`，然後重新啟動所有執行中的代理程式，現有程序不會自動擷取新憑證。

繼續操作前，請先停止 Creative Director (`Ctrl+C`)，因為 A2A 檢查器也會使用通訊埠 8000。

完成本機測試後，請停止 5 個專家伺服器 (在每個終端機中按下 `Ctrl+C`)。

12. 部署及測試專家代理程式

現在可以將代理程式部署到 Google Cloud。Cloud Run 是部署代理程式的絕佳服務。這項服務無伺服器、可擴充且容易使用。每個專家代理程式都會以獨立的 Cloud Run 服務形式部署。

Deployment 設定

每位專家的 Dockerfile 遵循以下模式：

```
FROM python:3.12-slim
WORKDIR /app
RUN apt-get update && apt-get install -y --no-install-recommends gcc curl

# Fast dependency install with uv
COPY --from=ghcr.io/astral-sh/uv:latest /uv /usr/local/bin/uv
COPY pyproject.toml .
RUN uv sync --no-install-project --no-dev

COPY . .
RUN useradd -m -u 1000 appuser && chown -R appuser:appuser /app
USER appuser

ENV PYTHONUNBUFFERED=1 PORT=8080 HOST=0.0.0.0
EXPOSE 8080
CMD ["uv", "run", "python", "agent.py"]
```

依序部署所有 5 位專家

```
cd ~/ai-creative-studio/workshop/starter
source .env

uv run deploy/deploy_all_specialists.py
```

這個指令碼會一次部署 5 個代理程式 (總共約需 10 到 12 分鐘)。循序部署可避免 Cloud Build 輪詢配額 (60 項要求/分鐘)。完成後，系統會將每個代理程式的 Cloud Run 網址寫回 `.env`。

部署 Designer 後，指令碼會自動在 GCS bucket 上授予 Cloud Run 服務帳戶 `roles/storage.objectCreator`，以便上傳產生的圖片。

如果您在 `.env` 中設定 Notion 憑證，指令碼也會將憑證安全地儲存在 **Secret Manager** 中 (如 `notion-token`、`notion-project-db-id`、`notion-tasks-db-id`)，並透過 `--set-secrets` 將憑證注入 Project Manager 服務，而非以純環境變數的形式。也就是說，權杖不會顯示在 Cloud Run 的環境分頁或 `gcloud` 指令記錄中。

注意：部署指令碼會使用 `--allow-unauthenticated`，讓每個服務都能公開存取，不必使用權杖。這是本研討會的刻意設計。在正式版本中，請移除這個旗標並新增以 IAM 為基礎的驗證。

驗證部署作業

部署完成後，指令碼會自動將 Cloud Run 網址寫回 `.env`，取代上一個步驟中的 localhost 網址：

```
source .env

echo "Deployed URLs:"
echo "  Brand Strategist: $STRATEGIST_AGENT_URL"
echo "  Copywriter:      $COPYWRITER_AGENT_URL"
echo "  Designer:         $DESIGNER_AGENT_URL"
echo "  Critic:           $CRITIC_AGENT_URL"
echo "  Project Manager:  $PM_AGENT_URL"
```

在下一個步驟中部署至 Agent Runtime 時，創意總監會自動使用這些 Cloud Run 網址。

驗證服務專員卡片

每個已部署的代理程式都會在 `/.well-known/agent.json` 顯示代理程式資訊卡。擷取這些項目，確認一切正常運作：

```
source .env

for agent_url in $STRATEGIST_AGENT_URL $COPYWRITER_AGENT_URL $DESIGNER_AGENT_URL
$CRITIC_AGENT_URL $PM_AGENT_URL; do
    echo "=== Agent Card: $agent_url ==="
    curl -s "${agent_url}/.well-known/agent.json" | python3 -m json.tool | grep -E
' "name" | "url" | "description" '
    echo ""
done
```

各代理程式的預期輸出內容：

```
"name": "brand_strategist",
"url": "https://brand-strategist-xxxx.run.app",
"description": "Brand strategist for market research and competitive insights"
```

使用 A2A 檢查器 (Cloud Run) 進行測試

您已在步驟 10 安裝 A2A 檢查器。啟動：

```
cd ~/a2a-inspector
bash scripts/run.sh
```

依序開啟「Web Preview」（網頁預覽）→「Change port」（變更通訊埠）→ `5001`。在連線欄位中輸入 Cloud Run 網址：

```
https://brand-strategist-xxxx.us-central1.run.app
```

按一下「連線」，由於服務是透過 `--allow-unauthenticated` 部署，因此不需要驗證權杖。

檢查工具會連線、驗證代理程式資訊卡，並讓您透過 A2A 進行互動式即時通訊。

每位專家都適用同樣的規定，只要輸入 Cloud Run 網址即可。

檢查部署至 Cloud Run 的代理程式

部署到 Cloud Run 後，請將檢查工具指向公開 HTTPS 網址，確認雲端部署作業是否正常運作：

5001-cs-682664025072-default.cs-europe-west1-iuzs.cloudshell.dev/?authuser=0

A2A Inspector

https://brand-strategist-ucrktqbhca-uc.a.run.app

► Authentication & Headers

▼ Agent Card

Agent card is valid.

```
},
"description": "Brand strategist for market research, trend analysis, and competitive insights",
"name": "brand_strategist",
"preferredTransport": "JSONRPC",
"protocolVersion": "0.3.0",
"skills": [
  {
    "description": "Brand strategist for market research, trend analysis, and competitive insights I am a Brand Strategist specializing in market research and trend analysis.\n\nIMPORTANT: Today's date is April 08, 2026.\n\nWhen conducting research, focus on current trends from 2026.\n\nUse search queries like \"[topic] trends 2026\" for recent insights.\n\nIMPORTANT: my role is RESEARCH ONLY. I do NOT create campaign content, captions, or designs.\n\nAfter providing research insights, my work is complete.\n\nmy expertise:\n- Identifying target audience insights and behaviors\n- Analyzing competitor strategies\n- Researching current social media trends\n- Understanding platform algorithms and best practices\n\nI have access to:\n- google_search: Search the web for competitors, trends, and market insights\n\nWhen given a campaign brief:\n1. Use google_search to research the target audience's current interests\n2. Search for and analyze 2-3 competitor brands\n3. Identify
```

工作流程完全相同，只要貼上 Cloud Run 網址、連線並傳送測試訊息即可。如果專員資訊卡載入且聊天室有回應，表示專員已正確部署且可聯絡。

13. 將創意總監部署至 Agent Runtime

自動調度管理工具會部署至 **Agent Runtime**，提供代管工作階段狀態、自動調整資源配置和內建追蹤功能。

為什麼自動調度管理工具需要 Agent Runtime ？

這五位專家會部署到 Cloud Run，他們是輕量級的無狀態專家，每人負責一項工作。創意總監有不同的需求：

規定	重要性解析
工作階段狀態	多步驟工作流程需要 45 秒以上。Agent Runtime 會維護自動調度工具呼叫之間的對話狀態，因此管道中途不會遺失任何內容。
變動負載	有時每小時一個廣告活動，有時則會同時執行多個廣告活動。閒置時，Agent Runtime 會將資源縮減至零，並自動擴充資源，因此您不必為閒置容量付費。
觀測能力	內建 Cloud Logging、Cloud Monitoring 和 Cloud Trace。您可以看到每通 A2A 呼叫、使用的每個權杖、每個延遲尖峰，不必新增任何檢測。
長時間執行的工作流程	Cloud Run 的要求逾時時間為 3600 秒。Agent Runtime 專為需要數分鐘的工作流程而設計，可管理重試作業和狀態持續性。

Cloud Run 是無狀態專家的合適平台。Agent Runtime 是有狀態自動調度管理工具的合適平台。

部署自動調度管理工具

開啟新的終端機執行這個步驟，然後重新匯出專案變數。在多個終端機之間作業後，殼層環境可能無法設定這些變數：

```
export PROJECT_ID=$(gcloud config get-value project)
export REGION="us-central1"
```

```
cd ~/ai-creative-studio/workshop/starter
source .env

uv run deploy/deploy_orchestrator.py --action deploy
```

這項作業需要約 5 到 10 分鐘。完成後，AGENT_ENGINE_ID 和 AGENT_ENGINE_RESOURCE_NAME 會儲存到 .env。

```
source .env
echo "Agent Engine ID: $AGENT_ENGINE_ID"
echo "Resource: $AGENT_ENGINE_RESOURCE_NAME"
```

部署方式

`client.agent_engines.create()` 會封裝 `App` 物件、連同依附元件上傳，並部署至受管理基礎架構。各項參數的用途如下：

```
import vertexai
from vertexai import Client, agent_engines

vertexai.init(project=PROJECT_ID, location=LOCATION, staging_bucket=STAGING_BUCKET)

# Wrap the App in an AdkApp adapter - enables tracing in Cloud Trace
adk_app = agent_engines.AdkApp(app=root_app, enable_tracing=True)

# Initialize client and deploy
client = Client(project=PROJECT_ID, location=LOCATION)

agent_engine_resource = client.agent_engines.create(
    agent=adk_app,
    config={
        "staging_bucket": STAGING_BUCKET, # GCS bucket for packaging artifacts
        "display_name": "Creative Director",
        # Python packages installed in the managed runtime - pin for reproducibility
        "requirements": [
            "google-cloud-aiplatform[agent_engines]>=1.132.0,<2.0.0",
            "google-adk[a2a]==1.31.1",
            "google-genai>=1.70.0",
            "google-cloud-storage>=2.10.0",
            "python-dotenv>=1.0.0",
            "pydantic>=2.0.0",
            "cloudpickle>=3.0.0",
        ],
        # Specialist URLs passed as env vars - the orchestrator reads these at runtime
        "env_vars": {
            "COPYWRITER_AGENT_URL": COPYWRITER_URL,
            "DESIGNER_AGENT_URL": DESIGNER_URL,
            "STRATEGIST_AGENT_URL": STRATEGIST_URL,
            "CRITIC_AGENT_URL": CRITIC_URL,
            "PM_AGENT_URL": PM_URL,
        },
    },
)

resource_name = agent_engine_resource.api_resource.name
agent_engine_id = resource_name.split("/")[-1]
```

幕後運作方式：

1. Agent Engine packages your App + requirements into a container
2. Uploads it to the staging bucket in your project
3. Deploys to managed compute (you never see or manage the VM)
4. Returns a resource name: `projects/.../locations/.../reasoningEngines/<id>`
5. That ID is saved to `.env` as `AGENT_ENGINE_ID`

部署完成後，協調器會透過環境變數中的網址，連線至五位 Cloud Run 專家

- 這些值會透過 `.env` 傳遞，然後才執行部署指令碼。
-

14. 執行端到端廣告活動

整個系統已部署完成。從 Agent Runtime 試用版執行完整廣告活動。

開啟 Agent Runtime 遊樂場

1. 前往 <https://console.cloud.google.com/agent-platform/runtimes>。您也可以依序點選「Agent Platform」>「Agents」>「Deployments」，前往 Agent Runtime。
2. 選取已部署的 Agent Runtime (creative-director)
3. 按一下左側邊欄中的「Playground」
4. 按一下「新對話」，開啟新的對話

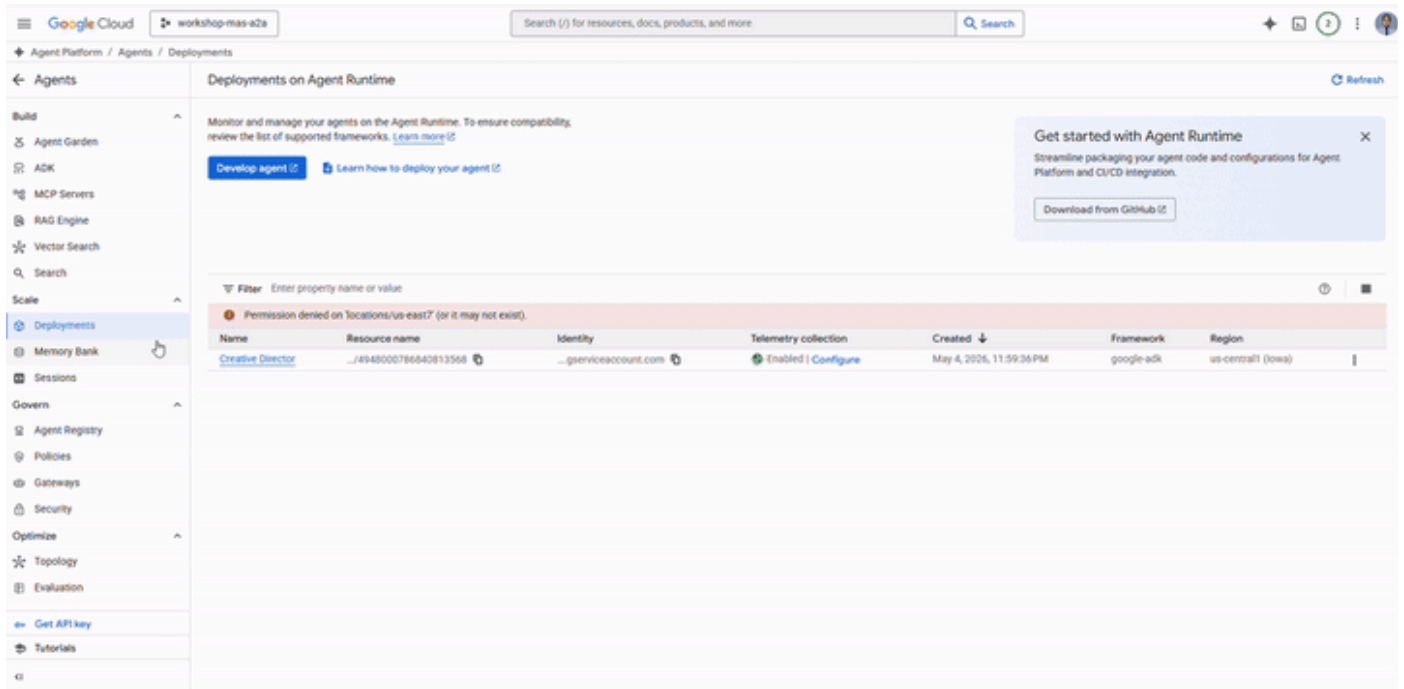
放送完整廣告活動

將簡報貼到對話中並傳送：

```
Create a complete Instagram campaign for:  
- Product: EcoFlow Smart Water Bottle (tracks hydration, keeps drinks cold 24h)  
- Target Audience: Health-conscious millennials, 25-35 years old  
- Platform: Instagram  
- Goal: Brand awareness + drive website traffic  
- Brand Voice: Motivational, clean, science-backed  
- Budget: $3,000  
- Timeline: Launch in 2 weeks
```

創意總監會依序執行所有 5 個代理程式：

1. 品牌策略師 → 市場調查、競爭對手分析、目標對象洞察
2. 文案人員 → 3 則附有說明文字、主題標記和行動號召的 Instagram 貼文
3. 設計師 → 透過 Gemini 生成每個貼文的視覺概念 + 實際圖片 (GCS URI)
4. 評論家 → 評分結果為「已核准」/「需要修訂」的品質審查
5. (視需要修訂) → 再次請文案撰寫人員或設計師提供意見
6. 專案經理 → 2 週時間表、工作細分、預算分配



測試單一代理程式轉送

在新工作階段中傳送較短的要求：

```
Research the luxury skincare market - top brands and trends in 2025
```

請注意，創意總監只會將此要求轉送給**品牌策略師**，不會呼叫其他服務專員。這是系統指令的請求分類邏輯，可正常運作。

檢查執行追蹤記錄

在控制台中執行下列操作：

1. 按一下左側邊欄中的「追蹤記錄」（在 Playground 旁邊）
2. 在「追蹤檢視畫面」下方，選取您剛執行的工作階段追蹤記錄
3. 展開追蹤樹狀結構，即可查看每個代理程式呼叫、輸入/輸出內容、延遲時間和權杖用量

每次與專家的 A2A 通話都會顯示為獨立的時距。您可以確切瞭解創意總監傳送給各個代理程式的內容，以及收到的回覆。

注意：如果「追蹤」分頁中未顯示工作階段，請稍候幾分鐘再重新整理頁面。

選用：從終端機執行

您也可以使用啟動器中已包含的 `run_campaign.py` 指令碼，以程式輔助方式執行廣告活動。

```
cd ~/ai-creative-studio/workshop/starter
uv run run_campaign.py
```

15. 清除

清理 Google Cloud 資源，以免產生後續費用。

執行拆解指令碼，讀取 `.env` 並刪除本程式碼研究室建立的所有項目：

```
bash deploy/teardown_gcp.sh
```

指令碼會顯示要刪除的內容，並在執行任何動作前提示您確認：

資源	系統會刪除的項目
Cloud Run 服務	品牌策略家、文案撰寫人員、設計師、評論家、專案經理
Agent Runtime	Creative Director 推論引擎 + 所有工作階段
Artifact Registry	<code>cloud-run-source-deploy</code> 存放區 + 所有 Docker 映像檔
GCS bucket	<code>{PROJECT_ID}-campaign-images</code> 、 <code>{PROJECT_ID}-agent-staging</code> 、 <code>run-sources-{PROJECT_ID}-{REGION}</code>
Secret Manager	<code>notion-token</code> 、 <code>notion-project-db-id</code> 、 <code>notion-tasks-db-id</code> (如果未建立，則會略過)

確認所有內容都已移除

```
gcloud run services list --region=us-central1
gcloud storage buckets list --project=$GCP_PROJECT_ID
```

預期輸出內容：空白清單或只有您自己的現有資源。

16. 摘要

恭喜！您已在 Google Cloud 上建構及部署正式環境等級的多代理 AI 系統。

您建構的內容

代理	功能	部署作業
品牌策略師	透過 Google 搜尋進行市場調查	Cloud Run
文案撰寫者	製作 Instagram 說明文字	Cloud Run
設計師	透過 Gemini 生成圖片 + 上傳至 GCS	Cloud Run
評論家	附有評分的商品審查	Cloud Run
專案經理	時間軸 + Notion MCP	Cloud Run
創意總監	透過 A2A 完整自動化調度管理	Agent Runtime

您學到的重要模式

1. **ADK Agent** - 定義具有指令和選用工具的 LLM 代理
2. **adk web** - 使用內建的即時通訊 UI，在本機執行及測試任何 ADK 代理
3. **SkillToolset** - 將可重複使用的知識封裝到隨選載入的模組化檔案中
4. **FunctionTool** - 將任何 Python 函式 (或外部模型) 封裝為可呼叫的代理程式工具
5. **to_a2a()** - 將任何 ADK 代理公開為符合 A2A 規範的 HTTPS 服務
6. **RemoteA2aAgent + AgentTool** - 將遠端代理程式調度為可呼叫的工具
7. **McpToolset** - 透過 MCP stdio 伺服器連線至外部服務
8. **EventsCompactionConfig** - 處理多代理程式長工作流程中的權杖限制
9. 結構化評論家輸出內容 - 機器可讀取的品質控管，可自動修訂
10. **Cloud Run** - 大規模部署容器化代理程式
11. **Agent Runtime** - 透過代管工作階段和追蹤功能，代管協調器

後續步驟

- 使用 `gemini-3.1-flash-image-preview` 的編輯功能，在設計工具中新增多輪圖像編輯功能
- 在 Cloud Run 服務中新增 **IAM 驗證** (移除 `--allow-unauthenticated`)
- 以 **LangGraph** 或 **CrewAI** 代理取代一位專家 - A2A 與框架無關
- 新增使用者意見回饋做為工具，讓參與者評估輸出內容並進行疊代
- 在 Cloud 控制台中探索 **Agent Runtime 追蹤記錄**

資源

- [Google ADK 說明文件](#)
 - [A2A 通訊協定規格](#)
 - [Agent Runtime 文件](#)
 - [ADK 程式碼研究室](#)
-